

Namespace Epson.RoboticsShared. ExtensionsAPI

Classes

[IRCXControllerAPI.ControllerSettingValue](#)

Represents a controller setting value.

[IRCXIOAPI.IRCXIOGroup.PortRange](#)

Represents a range of I/O port numbers.

[IRCXMainMenuItemProvider.MenuItem](#)

Represents a menu item.

[IRCXPointAPI.PointsChangedEventArgs](#)

Provides event data for a points change.

[IRCXPointAPI.RCXPointElement](#)

Represents a point element value.

[IRCXPointAPI.RCXPointFileDescriptor](#)

Descriptor for a points file.

[IRCXPreferencesAPI.PreferenceValue](#)

Represents a preference value for Preferences.

[IRCXProjectAPI.ProjectFileInfo](#)

Project file information.

[IRCXProjectAPI.ProjectPropertyValue](#)

Represents a setting value for project properties.

[IRCXProjectExplorerItemProvider.Item](#)

Represents a tree item description.

[IRCXProjectExplorerItemProvider.MenuItem](#)

Represents a context menu item description.

[IRCXRobotManagerAPI.RCXPartialRobotInfo](#)

Provides a partial subset of robot information.

[IRCXRobotManagerAPI.RobotSettingValue](#)

Represents the setting values related to robot configuration.

[IRCXWindowAPI.WebMessage](#)

Represents the common structure of messages exchanged with the web side.

[IRCXWindowAPI.WebViewInfo](#)

Contains definition and configuration details for a WebView component.

[RCXCaption](#)

Represents a caption specified by number or text.

[RCXCommon](#)

Common definitions for interfaces used by RC+ Extensions.

[RCXCommon.RCXSettingValue](#)

Base class for setting values.

Interfaces

[IRCXCaptionGetter](#)

Caption-related API

[IRCXConfiguration](#)

Configuration data access API for RC+ Extensions

[IRCXControllerAPI](#)

Controller API

[IRCXControllerConnectionAPI](#)

Controller Connection API

[IRCXControllerConnectionAPI.IRCXConnection](#)

Connection information.

[IRCXIOAPI](#)

Interface for I/O-related APIs.

[IRCXIOAPI.IRCXIOWatchGroup](#)

I/O group information.

[IRCXIOAPI.IRCXIOLabel](#)

I/O label object.

[IRCXIOAPI.IRCXIOLabel<T>](#)

Represents an I/O label that exposes typed read/write operations.

[IRCXIOAPI.IRCXIOWatcher](#)

I/O data monitoring object.

[IRCXIOAPI.IRCXInputIOLabel<T>](#)

I/O label for input ports.

[IRCXIOAPI.IRCXMemoryIOLabel<T>](#)

I/O label for memory I/O.

[IRCXIOAPI.IRCXOutputIOLabel<T>](#)

I/O label for output ports.

[IRCXLangRxCaption](#)

Language reactive caption

[IRCXMain](#)

RC+ Extensions Main

[IRCXMainMenuItemProvider](#)

Provides main menu items.

[IRCXPointAPI](#)

API related to points.

[IRCXPreferencesAPI](#)

API related to Preferences.

[IRCXProgramEditorAPI](#)

API related to the Program Editor.

[IRCXProgramEditorAPI.IRCXProgramEditor](#)

Represents the Program Editor.

[IRCXProgramExecutionAPI](#)

API related to program execution.

[IRCXProgramExecutionAPI.IRCXFunctionInfo](#)

Function information.

[IRCXProgramExecutionAPI.IRCXGlobalVariable](#)

Global variable object.

[IRCXProgramExecutionAPI.IRCXLocalVariable](#)

Local variable object.

[IRCXProgramExecutionAPI.IRCXModuleVariable](#)

Module variable object.

[IRCXProgramExecutionAPI.IRCXTask](#)

Represents a task object.

[IRCXProgramExecutionAPI.IRCXTask.IRCXCallStackEntry](#)

Represents a call stack entry.

[IRCXProgramExecutionAPI.IRCXVariable](#)

Variable object.

[IRCXProjectAPI](#)

Project-related API.

[IRCXProjectExplorerItemProvider](#)

Interface for providing items for the Project Explorer.

[IRCXProjectFileProvider](#)

Interface for providing additional files for a project.

[IRCXRobotManagerAPI](#)

Robot manager-related API

[IRCXRobotManagerAPI.IRCXJogger](#)

Represents a jog object.

[IRCXRobotManagerAPI.IRCXJointState](#)

Represents the joint state.

[IRCXRobotManagerAPI.IRCXRobotInfo](#)

Provides robot information.

[IRCXUserControlViewModel](#)

Interface for providing a view model for a UserControl hosted in a docking window.

[IRCXWindowAPI](#)

Provides window-related API.

Enums

[IRCXControllerConnectionAPI.IRCXConnection.CnKind](#)

The connection kind.

[IRCXControllerConnectionAPI.IRCXConnection.CnSeries](#)

The controller series.

[IRCXIOAPI.RCXIOKind](#)

The I/O kind.

[RCXMainMenuProvider.TopLevelMenu](#)

Specifies the top-level menu.

[RCXPointAPI.RCXPointElement.Elbow](#)

Specifies the elbow configuration.

[RCXPointAPI.RCXPointElement.Hand](#)

Specifies the hand orientation.

[RCXPointAPI.RCXPointElement.Wrist](#)

Specifies the wrist orientation.

[IRCXProgramExecutionAPI.IRCXTask.RCXTaskKind](#)

Represents the kind of a task.

[IRCXProgramExecutionAPI.IRCXTask.RCXTaskState](#)

Represents the state of a task.

[IRCXRobotManagerAPI.RCXJogCartesianAxis](#)

Jog Cartesian axis specification.

[IRCXRobotManagerAPI.RCXJogDistance](#)

Jog step amount (distance increment).

[IRCXRobotManagerAPI.RCXJogJointAxis](#)

Jog joint axis specification.

[IRCXRobotManagerAPI.RCXJogMode](#)

Jog mode specification.

[IRCXRobotManagerAPI.RCXJogSpeed](#)

Jog speed.

[IRCXWindowAPI.ButtonType](#)

Specifies the set of buttons to display in a message box.

[IRCXWindowAPI.ContentState](#)

Represents flags that indicate the current content state.

[IRCXWindowAPI.IconType](#)

Specifies the icon to display in a message box.

[IRCXWindowAPI.ResponseType](#)

Represents the user's response in a message box.

[RCXCaption.TypeTag](#)

Specifies how the caption is defined.

[RCXCommon.RCXResult](#)

Common result codes.

Delegates

[IRCXIOAPI.RCXIODataChangedCallback](#)

Callback invoked when I/O data changes.

[IRCXPointAPI.PointsChangedEventHandler](#)

Represents the delegate for handling points changed events.

[RCXExternalFunction](#)

Delegate for invoking an external function.

Interface IRCXCaptionGetter

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Caption-related API

```
public interface IRCXCaptionGetter
```

Properties

CurrentLanguage

Gets the currently selected display language.

```
string CurrentLanguage { get; }
```

Property Value

[string](#)[↗]

A [string](#)[↗] containing the language display name.

this[int]

Gets the caption associated with the specified number.

```
string this[int number] { get; }
```

Parameters

number [int](#)[↗]

The caption number used to access the item.

Property Value

[string](#)

The caption text.

this[string]

Gets the caption associated with the specified name.

```
IRCXLangRxCaption this[string name] { get; }
```

Parameters

name [string](#)

The name of the caption to retrieve. Cannot be null.

Property Value

[IRCXLangRxCaption](#)

The caption object associated with the specified name.

LanguageChanged

Gets or sets the handler that is invoked when the display language changes.

```
Action? LanguageChanged { get; set; }
```

Property Value

[Action](#)

An [Action](#) delegate that is called upon a language change.

NameToNumber

Gets or sets the function used to convert a name to its corresponding number.


```
Func<string, int?>? NameToNumber { get; set; }
```

Property Value

[Func](#) <[string](#), [int](#)?>

Remarks

The function should accept a string representing a name and return the associated number as an integer, or null if the name does not correspond to a number. This property can be set to customize how names are mapped to numbers within the application.

SupportedLanguages

Gets the list of supported languages.

```
IReadOnlyList<string> SupportedLanguages { get; }
```

Property Value

[IReadOnlyList](#) <[string](#)>

Interface IRCXConfiguration

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Configuration data access API for RC+ Extensions

```
public interface IRCXConfiguration
```

Methods

Read(bool)

Reads the settings data.

```
byte[]? Read(bool global = true)
```

Parameters

global [bool](#) 

Indicates whether to read machine-wide settings shared by all Windows users (**true**) or user-specific settings for the current Windows user (**false**).

Returns

[byte](#)  []

The settings data as a byte array, or **null** if the data is not stored or an error occurs.

Write(byte[], bool)

Writes the settings data.

```
bool Write(byte[] data, bool global = true)
```

Parameters

data [byte](#)[]

The settings data to write.

global [bool](#)

Indicates whether to write machine-wide settings shared by all Windows users (**true**) or user-specific settings for the current Windows user (**false**).

Returns

[bool](#)

true if the write operation succeeds; otherwise, **false**.

Interface IRCXControllerAPI

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Controller API

```
public interface IRCXControllerAPI : INotifyPropertyChanged
```

Inherited Members

[INotifyPropertyChanged.PropertyChanged](#) 

Properties

IsEmergencyStopped

Indicates whether the system is in an emergency stop state.

```
bool? IsEmergencyStopped { get; }
```

Property Value

[bool](#) ?

true if the system is emergency-stopped; **false** if it is not; **null** if the state is unknown.

IsMotorOn

Indicates whether the motor is on.

```
bool? IsMotorOn { get; }
```

Property Value

[bool](#) ?

true if the motor is on; **false** if it is off; **null** if the state is unknown.

IsPowerHigh

Indicates whether the power level is high.

```
bool? IsPowerHigh { get; }
```

Property Value

[bool](#)?

true if the power level is high; **false** if it is not high; **null** if the state is unknown.

IsSafeguardOpened

Indicates whether the safeguard is open.

```
bool? IsSafeguardOpened { get; }
```

Property Value

[bool](#)?

true if the safeguard is open; **false** if it is closed; **null** if the state is unknown.

IsSystemErrorOccurred

Indicates whether a system error has occurred.

```
bool? IsSystemErrorOccurred { get; }
```

Property Value

[bool](#)?

true if a system error has occurred; **false** if no error is present; **null** if the state is unknown.

Methods

CancelSetControllerSettings(string)

Cancels a controller settings set session.

```
RCXCommon.RCXResult CancelSetControllerSettings(string sessionId)
```

Parameters

sessionId [string](#)

The session ID to cancel.

Returns

[RCXCommon.RCXResult](#)

The operation result.

CommitSetControllerSettingsAsync(string)

Asynchronously commits the controller settings set.

```
Task<RCXCommon.RCXResult> CommitSetControllerSettingsAsync(string sessionId)
```

Parameters

sessionId [string](#)

The session ID to commit.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the operation result.

Remarks

The controller may restart after the commit.

GetAvailableOptionNames()

Retrieves the names of available controller options.

```
(RCXCommon.RCXResult, IEnumerable<string>) GetAvailableOptionNames()
```

Returns

([RCXCommon.RCXResult](#), [IEnumerable](#) <[string](#)>)

A tuple containing the operation result and the collection of available option names.

GetControllerSettings(string?, string)

Retrieves controller settings for the specified category.

```
(RCXCommon.RCXResult, IDictionary<string, IRCXControllerAPI.ControllerSettingValue>?)  
GetControllerSettings(string? sessionId, string categoryName)
```

Parameters

sessionId [string](#)

The session ID. Pass **null** before starting an update session (no active session).

categoryName [string](#)

The category name for which to retrieve settings.

Returns

([RCXCommon.RCXResult](#), [IDictionary](#) <[string](#), [IRCXControllerAPI.ControllerSettingValue](#)>)

A tuple containing the operation result and a dictionary of setting values, or **null** if no settings are stored or an error occurs.

GetControllerSettingsCategoryNames()

Retrieves the controller setting category names.

```
IEnumerable<string> GetControllerSettingsCategoryNames()
```

Returns

[IEnumerable](#) <[string](#)>

A collection of category names for the controller settings.

IsOptionEnabled(string)

Determines whether the specified controller option is enabled.

```
(RCXCommon.RCXResult, bool) IsOptionEnabled(string optionName)
```

Parameters

optionName [string](#)

The name of the option to check.

Returns

([RCXCommon.RCXResult](#), [bool](#))

A tuple containing the operation result and a flag indicating whether the option is enabled.

MotorOffAsync()

Asynchronously turns the motor off.

```
Task<RCXCommon.RCXResult> MotorOffAsync()
```

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns an [RCXCommon.RCXResult](#) indicating the operation outcome.

MotorOnAsync()

Asynchronously turns the motor on.

```
Task<RCXCommon.RCXResult> MotorOnAsync()
```

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns an [RCXCommon.RCXResult](#) indicating the operation outcome.

MotorOnOffAsync(bool)

Asynchronously turns the motor on or off.

```
Task<RCXCommon.RCXResult> MotorOnOffAsync(bool turnOn)
```

Parameters

turnOn [bool](#)

Set to **true** to turn the motor on; set to **false** to turn it off.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns an [RCXCommon.RCXResult](#) indicating the operation outcome.

PowerHighAsync()

Asynchronously sets the power to high.

```
Task<RCXCommon.RCXResult> PowerHighAsync()
```

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns an [RCXCommon.RCXResult](#) indicating the operation outcome.

PowerHighLowAsync(bool)

Asynchronously switches the power between high and low.

```
Task<RCXCommon.RCXResult> PowerHighLowAsync(bool turnHigh)
```

Parameters

turnHigh [bool](#)

Set to **true** to switch to high power; set to **false** to switch to low power.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns an [RCXCommon.RCXResult](#) indicating the operation outcome.

PowerLowAsync()

Asynchronously sets the power to low.

```
Task<RCXCommon.RCXResult> PowerLowAsync()
```

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns an [RCXCommon.RCXResult](#) indicating the operation outcome.

ResetAsync()

Asynchronously resets the controller.

```
Task<RCXCommon.RCXResult> ResetAsync()
```

Returns

[Task](#)  <[RCXCommon.RCXResult](#)>

A task that returns an [RCXCommon.RCXResult](#) indicating the operation outcome.

SetControllerSettingsAsync(string, string, IDictionary<string, ControllerSettingValue>)

Asynchronously applies controller settings for the specified category within the active update session.

```
Task<RCXCommon.RCXResult> SetControllerSettingsAsync(string sessionId, string categoryName,  
IDictionary<string, IRCXControllerAPI.ControllerSettingValue> settings)
```

Parameters

sessionId [string](#) 

The active settings update session ID.

categoryName [string](#) 

The category name to update.

settings [IDictionary](#)  <[string](#) , [IRCXControllerAPI.ControllerSettingValue](#)>

A dictionary of setting key–value pairs to apply. Keys are setting identifiers; values are [IRCXControllerAPI.ControllerSettingValue](#) instances.

Returns

[Task](#)  <[RCXCommon.RCXResult](#)>

A task that returns the operation result.

StartSetControllerSettingsAsync()

Asynchronously starts a controller settings set session.

```
Task<(RCXCommon.RCXResult, string?)> StartSetControllerSettingsAsync()
```

Returns

[Task](#) <[RCXCommon.RCXResult](#), [string](#)>

A task that returns a tuple containing the operation result and the created session ID, or `null` for the session ID if session creation failed.

Class IRCXControllerAPI.ControllerSettingValue

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents a controller setting value.

```
public class IRCXControllerAPI.ControllerSettingValue : RCXCommon.RCXSettingValue
```

Inheritance

[object](#)  ← [RCXCommon.RCXSettingValue](#) ← IRCXControllerAPI.ControllerSettingValue

Inherited Members

[RCXCommon.RCXSettingValue.Value](#) , [RCXCommon.RCXSettingValue.ValueType](#) ,
[RCXCommon.RCXSettingValue.CanSet](#) , [RCXCommon.RCXSettingValue.Choices](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Interface IRCXControllerConnectionAPI

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Controller Connection API

```
public interface IRCXControllerConnectionAPI : INotifyPropertyChanged
```

Inherited Members

[INotifyPropertyChanged.PropertyChanged](#) 

Properties

Connections

Gets the collection of available connection entries.

```
IEnumerable<IRCXControllerConnectionAPI.IRCXConnection> Connections { get; }
```

Property Value

[IEnumerable](#)  <[IRCXControllerConnectionAPI.IRCXConnection](#)>

A collection of [IRCXControllerConnectionAPI.IRCXConnection](#) instances representing available connections.

IsAutoConnectEnabled

Gets a value indicating whether automatic connection is enabled.

```
bool? IsAutoConnectEnabled { get; }
```

Property Value

[bool](#) ?

`true` if auto-connect is enabled; `false` if it is disabled; `null` if the state cannot be retrieved.

IsMonitorMode

Gets a value indicating whether the controller is in monitor mode.

```
bool IsMonitorMode { get; }
```

Property Value

[bool](#)

`true` if the controller is in monitor mode; `false` otherwise.

IsOfflineMode

Gets a value indicating whether the system is in offline mode.

```
bool? IsOfflineMode { get; }
```

Property Value

[bool](#)?

`true` if offline mode is enabled; `false` if it is disabled; `null` if the state cannot be retrieved.

IsOnline

Gets a value indicating whether the controller is online.

```
bool? IsOnline { get; }
```

Property Value

[bool](#)?

`true` if the controller is online; `false` if it is offline; `null` if the state cannot be determined (e.g., during connection or disconnection).

Methods

AddConnectionAsync(string, CnKind, CnSeries, string?)

Asynchronously adds new connection information.

```
Task<RCXCommon.RCXResult> AddConnectionAsync(string name,  
IRCXControllerConnectionAPI.IRCXConnection.CnKind kind,  
IRCXControllerConnectionAPI.IRCXConnection.CnSeries series, string? ipAddress = null)
```

Parameters

name [string](#)

name for the connection.

kind [IRCXControllerConnectionAPI.IRCXConnection.CnKind](#)

The connection kind.

series [IRCXControllerConnectionAPI.IRCXConnection.CnSeries](#)

The controller series.

ipAddress [string](#)

The IP address to use, if applicable; otherwise `null`.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that completes with an [RCXCommon.RCXResult](#) indicating the outcome.

ConnectControllerAsync(int?, bool)

Asynchronously connects to the controller.


```
Task<bool> ConnectControllerAsync(int? connectionNumber = null, bool prohibitMonitorMode  
= true)
```

Parameters

connectionNumber [int](#)?

The number of the controller to connect to. If `null`, it connects to the last connected controller.

prohibitMonitorMode [bool](#)

Indicates whether connections in monitor mode are disallowed. When `true`, a monitor-mode connection is prohibited.

Returns

[Task](#) <[bool](#)>

A task that returns `true` if the connection completes successfully; otherwise, `false`.

CopyVirtualConnectionAsync(string, string)

Asynchronously creates and adds a copy of the connection information for a virtual controller connection.

```
Task<RCXCommon.RCXResult> CopyVirtualConnectionAsync(string name, string sourceName)
```

Parameters

name [string](#)

The name to assign to the copied connection.

sourceName [string](#)

The name of the source virtual connection to copy from.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that completes with an [RCXCommon.RCXResult](#) indicating the outcome.

DeleteConnectionAsync(string)

Asynchronously deletes the specified connection information.

```
Task<RCXCommon.RCXResult> DeleteConnectionAsync(string name)
```

Parameters

name [string](#) 

The name of the connection to delete.

Returns

[Task](#)  <[RCXCommon.RCXResult](#)>

A task that completes with an [RCXCommon.RCXResult](#) indicating the outcome.

Remarks

USB connections and connection information of controllers that are currently connected cannot be deleted.

DisconnectControllerAsync()

Disconnects from the controller.

```
Task DisconnectControllerAsync()
```

Returns

[Task](#) 

A task that represents the asynchronous disconnect operation.

GetLastConnection()

Retrieves the connection information of the most recent connection.

```
IRCXControllerConnectionAPI.IRCXConnection? GetLastConnection()
```

Returns

[IRCXControllerConnectionAPI.IRCXConnection](#)

The [IRCXControllerConnectionAPI.IRCXConnection](#) describing the last connection; otherwise `null` if it cannot be determined.

Remarks

The returned information may differ from the original connection settings if changes were made after the disconnect.

ModifyEthernetConnectionAsync(string, string, CnSeries, string)

Asynchronously updates the connection information for an Ethernet connection.

```
Task<RCXCommon.RCXResult> ModifyEthernetConnectionAsync(string name, string newName,  
IRCXControllerConnectionAPI.IRCXConnection.CnSeries newSeries, string newIpAddress)
```

Parameters

name [string](#) 

The current name of the connection.

newName [string](#) 

The new name to assign to the connection.

newSeries [IRCXControllerConnectionAPI.IRCXConnection.CnSeries](#)

The new controller series to set.

newIpAddress [string](#) 

The new IP address to set.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that completes with an [RCXCommon.RCXResult](#) indicating the outcome.

Remarks

Connection information of a controller that is currently connected cannot be updated.

ModifyVirtualConnectionAsync(string, string)

Asynchronously updates the connection information for a virtual controller connection.

```
Task<RCXCommon.RCXResult> ModifyVirtualConnectionAsync(string name, string newName)
```

Parameters

name [string](#)

The current name of the connection.

newName [string](#)

The new name to assign to the connection.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that completes with an [RCXCommon.RCXResult](#) indicating the outcome.

Remarks

Connection information of a controller that is currently connected cannot be updated.

SetAutoConnectEnabledAsync(bool)

Asynchronously enables or disables auto-connect.

```
Task<RCXCommon.RCXResult> SetAutoConnectEnabledAsync(bool enabled)
```

Parameters

enabled [bool](#)

[true](#) to enable auto-connect; [false](#) to disable it.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that completes with an [RCXCommon.RCXResult](#) indicating the outcome.

SetOfflineModeAsync(bool)

Asynchronously enables or disables offline mode.

```
Task<RCXCommon.RCXResult> SetOfflineModeAsync(bool enabled)
```

Parameters

enabled [bool](#)

[true](#) to enable offline mode; [false](#) to disable it.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that completes with an [RCXCommon.RCXResult](#) indicating the outcome.

Interface IRCXControllerConnectionAPI. IRCXConnection

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Connection information.

```
public interface IRCXControllerConnectionAPI.IRCXConnection
```

Properties

IPAddress

Gets the IP address when the connection kind is Ethernet.

```
string? IPAddress { get; }
```

Property Value

[string](#)

The IPv4 or IPv6 address as a [string](#), or `null` when the connection kind is not [Ethernet](#).

IsPasswordProtected

Gets a value indicating whether the connection is protected by a password.

```
bool IsPasswordProtected { get; }
```

Property Value

[bool](#)

`true` if password protection is enabled; otherwise, `false`.

Kind

Gets the kind of the connection.

```
IRCXControllerConnectionAPI.IRCXConnection.CnKind Kind { get; }
```

Property Value

[IRCXControllerConnectionAPI.IRCXConnection.CnKind](#)

A [IRCXControllerConnectionAPI.IRCXConnection.CnKind](#) value indicating the connection type.

Name

Gets the display name of the connection.

```
string Name { get; }
```

Property Value

[string](#)

A [string](#) representing the connection name.

Number

Gets the number assigned to the connection.

```
int Number { get; }
```

Property Value

[int](#)

The implementation-defined connection number.

Series

Gets the controller series of the target.

```
IRCXControllerConnectionAPI.IRCXConnection.CnSeries Series { get; }
```

Property Value

[IRCXControllerConnectionAPI.IRCXConnection.CnSeries](#)

A [IRCXControllerConnectionAPI.IRCXConnection.CnSeries](#) value representing the controller series.

Enum IRCXControllerConnectionAPI. IRCXConnection.CnKind

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

The connection kind.

```
public enum IRCXControllerConnectionAPI.IRCXConnection.CnKind
```

Fields

Ethernet = 2

Ethernet connection.

USB = 1

USB connection.

Virtual = 0

Virtual connection.

Enum IRCXControllerConnectionAPI. IRCXConnection.CnSeries

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

The controller series.

```
public enum IRCXControllerConnectionAPI.IRCXConnection.CnSeries
```

Fields

RC700 = 2

RC700/90/T/VT series.

RC800 = 1

RC800 series.

Undefined = 0

Undefined.

Interface IRCXIOAPI

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Interface for I/O-related APIs.

```
public interface IRCXIOAPI : INotifyPropertyChanged
```

Inherited Members

[INotifyPropertyChanged.PropertyChanged](#) 

Properties

IOLabels

Gets a read-only list of I/O labels.

```
ICollection<IRCXIOAPI.IRCXIOLabel> IOLabels { get; }
```

Property Value

[ICollection](#)  <[IRCXIOAPI.IRCXIOLabel](#)>

A read-only list of [IRCXIOAPI.IRCXIOLabel](#) objects.

Methods

AddIOLabel(string, RCXIOKind, Type, int, string?)

Adds or updates an I/O label.

```
RCXCommon.RCXResult AddIOLabel(string label, IRCXIOAPI.RCXIOKind kind, Type dataType, int port, string? comment)
```

Parameters

label [string](#)

The name of the label.

kind [IRCXIOAPI.RCXIOKind](#)

The I/O kind.

dataType [Type](#)

The data type of the I/O ports (supported: [bool](#), [byte](#), [ushort](#)).

port [int](#)

The port number.

comment [string](#)

The comment for the label.

Returns

[RCXCommon.RCXResult](#)

An [RCXCommon.RCXResult](#) indicating the result of the operation.

Remarks

If the specified label name already exists, it will be updated.

CreateWatcher<T>(RCXIOKind, int, RCXIODataChangedCallback)

Creates a watcher for the specified I/O kind and port and registers a data change callback.

```
(RCXCommon.RCXResult, IRCXIOAPI.IRCXIOWatcher?) CreateWatcher<T>(IRCXIOAPI.RCXIOKind kind,
int port, IRCXIOAPI.RCXIODataChangedCallback callback)
```

Parameters

kind [IRCXIOAPI.RCXIOKind](#)

The I/O kind to watch.

port [int](#)

The port number to watch.

callback [IRCXIOAPI.IRCXIOWatcher](#)

The callback to invoke when the data changes.

Returns

([RCXCommon.RCXResult](#), [IRCXIOAPI.IRCXIOWatcher](#))

A tuple containing an [RCXCommon.RCXResult](#) indicating the result of the operation and the created [IRCXIOAPI.IRCXIOWatcher](#) instance; the watcher is [null](#) if the operation fails.

Type Parameters

T

The data type associated with the I/O port (e.g., [bool](#), [byte](#), [ushort](#)).

DeleteIOLabel(string)

Deletes an I/O label.

```
RCXCommon.RCXResult DeleteIOLabel(string label)
```

Parameters

label [string](#)

The label name to delete.

Returns

[RCXCommon.RCXResult](#)

An [RCXCommon.RCXResult](#) indicating the result of the operation.

GetIOGroups()

Retrieves the list of I/O group information.

```
(RCXCommon.RCXResult, IEnumerable<IRCXIOAPI.IRCXIOGroup>) GetIOGroups()
```

Returns

([RCXCommon.RCXResult](#), [IEnumerable](#) <[IRCXIOAPI.IRCXIOGroup](#)>)

A tuple containing an [RCXCommon.RCXResult](#) indicating the result of the operation and an enumerable of [IRCXIOAPI.IRCXIOGroup](#) instances.

IOReadInput<T>(int)

Reads data from the specified input port.

```
(RCXCommon.RCXResult, T) IOReadInput<T>(int port)
```

Parameters

port [int](#)

The input port number.

Returns

([RCXCommon.RCXResult](#), T)

A tuple containing an [RCXCommon.RCXResult](#) indicating the result of the operation and the read value of type **T**. The value is meaningful only if the result indicates success.

Type Parameters

T

The data type to read (supported: [bool](#), [byte](#), [ushort](#)).

IOReadOutput<T>(int)

Reads data from the specified output port.

`(RCXCommon.RCXResult, T) IOReadOutput<T>(int port)`

Parameters

port [int](#)

The output port number.

Returns

[RCXCommon.RCXResult](#), T)

A tuple containing an [RCXCommon.RCXResult](#) indicating the result of the operation and the read value of type T. The value is meaningful only if the result indicates success.

Type Parameters

T

The data type to read (supported: [bool](#), [byte](#), [ushort](#)).

IOWriteInput<T>(int, T)

Writes data to the specified input port.

`RCXCommon.RCXResult IOWriteInput<T>(int port, T data)`

Parameters

port [int](#)

The input port number.

data T

The data to write.

Returns

[RCXCommon.RCXResult](#)

An [RCXCommon.RCXResult](#) indicating the outcome of the operation.

Type Parameters

T

The data type to write (supported: [bool](#), [byte](#), [ushort](#)).

Remarks

Effective only for virtual I/O.

IOWriteOutput<T>(int, T)

Writes data to the specified output port.

```
RCXCommon.RCXResult IOWriteOutput<T>(int port, T data)
```

Parameters

port [int](#)

The output port number.

data T

The data to write.

Returns

[RCXCommon.RCXResult](#)

An [RCXCommon.RCXResult](#) indicating the outcome of the operation.

Type Parameters

T

The data type to write (supported: [bool](#), [byte](#), [ushort](#)).

MemoryIORead<T>(int)

Reads data from the specified memory I/O port.

```
(RCXCommon.RCXResult, T) MemoryIORead<T>(int port)
```

Parameters

port [int](#)

The memory I/O port number.

Returns

([RCXCommon.RCXResult](#), T)

A tuple containing an [RCXCommon.RCXResult](#) indicating the result of the operation and the read value of type `T`. The value is meaningful only if the result indicates success.

Type Parameters

`T`

The data type to read (supported: [bool](#), [byte](#), [ushort](#)).

MemoryIOWrite<T>(int, T)

Writes data to the specified memory I/O port.

```
RCXCommon.RCXResult MemoryIOWrite<T>(int port, T data)
```

Parameters

port [int](#)

The memory I/O port number.

data `T`

The data to write.

Returns

[RCXCommon.RCXResult](#)

An [RCXCommon.RCXResult](#) indicating the result of the operation.

Type Parameters

T

The data type to write (supported: [bool](#)[↗], [byte](#)[↗], [ushort](#)[↗]).

Interface IRCXIOAPI.IRCXIOGroup

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

I/O group information.

```
public interface IRCXIOAPI.IRCXIOGroup
```

Properties

Name

Gets the display name of the I/O group.

```
string Name { get; }
```

Property Value

[string](#)

Group name.

Methods

GetInputPortRange<T>()

Gets the valid input port number range for the specified data type.

```
IRCXIOAPI.IRCXIOGroup.PortRange? GetInputPortRange<T>()
```

Returns

[IRCXIOAPI.IRCXIOGroup.PortRange](#)

A [IRCXIOAPI.IRCXIOWGroup.PortRange](#) describing the valid input port numbers for **T**, or [null](#) if the type is not supported.

Type Parameters

T

The data type of the input ports (supported: [bool](#), [byte](#), [ushort](#)).

GetOutputPortRange<T>()

Gets the valid output port number range for the specified data type.

```
IRCXIOAPI.IRCXIOWGroup.PortRange? GetOutputPortRange<T>()
```

Returns

[IRCXIOAPI.IRCXIOWGroup.PortRange](#)

A [IRCXIOAPI.IRCXIOWGroup.PortRange](#) describing the valid output port numbers for **T**, or [null](#) if the type is not supported.

Type Parameters

T

The data type of the output ports (supported: [bool](#), [byte](#), [ushort](#)).

ReadInput<T>(int)

Reads data from the specified input port.

```
(RCXCommon.RCXResult, T) ReadInput<T>(int port)
```

Parameters

port [int](#)

The input port number.

Returns

([RCXCommon.RCXResult](#), T)

A tuple containing an [RCXCommon.RCXResult](#) indicating the result of the operation and the read value of type **T**. The value is meaningful only if the result indicates success.

Type Parameters

T

The data type to read (supported: [bool](#), [byte](#), [ushort](#)).

ReadOutput<T>(int)

Reads data from the specified output port.

([RCXCommon.RCXResult](#), T) ReadOutput<T>([int](#) port)

Parameters

port [int](#)

The output port number.

Returns

([RCXCommon.RCXResult](#), T)

A tuple containing an [RCXCommon.RCXResult](#) indicating the result of the operation and the read value of type **T**. The value is meaningful only if the result indicates success.

Type Parameters

T

The data type to read (supported: [bool](#), [byte](#), [ushort](#)).

WriteInput<T>(int, T)

Writes data to the specified input port.

```
RCXCommon.RCXResult WriteInput<T>(int port, T data)
```

Parameters

port [int](#)

The input port number.

data T

The data to write.

Returns

[RCXCommon.RCXResult](#)

An [RCXCommon.RCXResult](#) indicating the outcome of the operation.

Type Parameters

T

The data type to write (supported: [bool](#), [byte](#), [ushort](#)).

Remarks

Effective only for virtual I/O.

WriteOutput<T>(int, T)

Writes data to the specified output port.

```
RCXCommon.RCXResult WriteOutput<T>(int port, T data)
```

Parameters

port [int](#)

The output port number.

data T

The data to write.

Returns

[RCXCommon.RCXResult](#)

An [RCXCommon.RCXResult](#) indicating the outcome of the operation.

Type Parameters

T

The data type to write (supported: [bool](#)[↗], [byte](#)[↗], [ushort](#)[↗]).

Class IRCXIOAPI.IRCXIOGroup.PortRange

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents a range of I/O port numbers.

```
public record IRCXIOAPI.IRCXIOGroup.PortRange : IEquatable<IRCXIOAPI.IRCXIOGroup.PortRange>
```

Inheritance

[object](#)  ← IRCXIOAPI.IRCXIOGroup.PortRange

Implements

[IEquatable](#)  <[IRCXIOAPI.IRCXIOGroup.PortRange](#)>

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

PortRange(int, int)

Represents a range of I/O port numbers.

```
public PortRange(int startPort, int endPort)
```

Parameters

startPort [int](#) 

The first (lowest) port number in the range.

endPort [int](#) 

The last (highest) port number in the range.

Properties

endPort

The last (highest) port number in the range.

```
public int endPort { get; init; }
```

Property Value

[int](#)

startPort

The first (lowest) port number in the range.

```
public int startPort { get; init; }
```

Property Value

[int](#)

Interface IRCXIOAPI.IRCXIOLabel

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

I/O label object.

```
public interface IRCXIOAPI.IRCXIOLabel
```

Properties

Description

Gets an optional description for the label.

```
string Description { get; }
```

Property Value

[string](#)

Free-form descriptive text; may be empty.

Label

Gets the label assigned to the I/O port.

```
string Label { get; }
```

Property Value

[string](#)

The label.

Port

Gets the I/O port number.

```
int Port { get; }
```

Property Value

[int](#)

The port number.

Methods

CreateWatcher(RCXIODataChangedCallback)

Creates a watcher and registers a data change callback.

```
(RCXCommon.RCXResult, IRCXIOAPI.IRCXIOWatcher?)  
CreateWatcher(IRCXIOAPI.RCXIODataChangedCallback callback)
```

Parameters

callback [IRCXIOAPI.RCXIODataChangedCallback](#)

The callback to invoke when the I/O data changes.

Returns

[\(RCXCommon.RCXResult, IRCXIOAPI.IRCXIOWatcher\)](#)

A tuple containing an [RCXCommon.RCXResult](#) indicating the result of the operation and the created [IRCXIOAPI.IRCXIOWatcher](#) instance; the watcher is [null](#) if the operation fails.

GetDataTypeName()

Retrieves the name of the data type.

`string? GetDataTypeName()`

Returns

[string](#) 

A string representing the data type name, or [null](#)  if unavailable.

Remarks

The value is one of: "Bit", "Byte", or "Word".

GetKind()

Retrieves the I/O kind.

`IRCXIOAPI.RCXIOKind GetKind()`

Returns

[IRCXIOAPI.RCXIOKind](#)

An [IRCXIOAPI.RCXIOKind](#) value indicating the type of I/O.

Interface IRCXIOAPI.IRCXIOLabel<T>

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents an I/O label that exposes typed read/write operations.

```
public interface IRCXIOAPI.IRCXIOLabel<T> : IRCXIOAPI.IRCXIOLabel
```

Type Parameters

T

The data type associated with the label (supported: [bool](#)[↗], [byte](#)[↗], [ushort](#)[↗]).

Inherited Members

[IRCXIOAPI.IRCXIOLabel.Label](#) , [IRCXIOAPI.IRCXIOLabel.Description](#) , [IRCXIOAPI.IRCXIOLabel.Port](#) , [IRCXIOAPI.IRCXIOLabel.GetDataTypeName\(\)](#) , [IRCXIOAPI.IRCXIOLabel.GetKind\(\)](#) , [IRCXIOAPI.IRCXIOLabel.CreateWatcher\(IRCXIOAPI.RCXIODataChangedCallback\)](#)

Methods

Read()

Reads the data.

```
(RCXCommon.RCXResult, T) Read()
```

Returns

([RCXCommon.RCXResult](#), T)

A tuple containing the [RCXCommon.RCXResult](#) and the read value of type **T**.

Write(T)

Writes the data.

RCXCommon.RCXResult **Write**(T data)

Parameters

data T

The value to write.

Returns

[RCXCommon.RCXResult](#)

The result code.

Interface IRCXIOAPI.IRCXIOWatcher

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

I/O data monitoring object.

```
public interface IRCXIOAPI.IRCXIOWatcher : IDisposable
```

Inherited Members

[IDisposable.Dispose\(\)](#) 

Properties

IsWatching

Gets or sets whether the watcher is currently monitoring I/O data.

```
bool IsWatching { get; set; }
```

Property Value

[bool](#) 

true if monitoring is active; otherwise, **false**.

Port

Gets the I/O port number being monitored.

```
int Port { get; }
```

Property Value

[int](#) 

The target I/O port number.

Methods

GetDataTypeName()

Retrieves the name of the data type.

```
string? GetDataTypeName()
```

Returns

[string](#) 

A string representing the data type name, or [null](#)  if unavailable.

Remarks

The value is one of: "Bit", "Byte", or "Word".

GetKind()

Retrieves the I/O kind.

```
IRCXIOAPI.RCXIOKind GetKind()
```

Returns

[IRCXIOAPI.RCXIOKind](#)

An [IRCXIOAPI.RCXIOKind](#) value indicating the type of I/O.

Interface IRCXIOAPI.IRCXInputIOLabel<T>

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

I/O label for input ports.

```
public interface IRCXIOAPI.IRCXInputIOLabel<T> : IRCXIOAPI.IRCXIOLabel<T>,
    IRCXIOAPI.IRCXIOLabel
```

Type Parameters

T

Data type (bool / byte / ushort).

Inherited Members

[IRCXIOAPI.IRCXIOLabel<T>.Read\(\)](#) , [IRCXIOAPI.IRCXIOLabel<T>.Write\(T\)](#) , [IRCXIOAPI.IRCXIOLabel.Label](#) ,
[IRCXIOAPI.IRCXIOLabel.Description](#) , [IRCXIOAPI.IRCXIOLabel.Port](#) ,
[IRCXIOAPI.IRCXIOLabel.GetDataTypeName\(\)](#) , [IRCXIOAPI.IRCXIOLabel.GetKind\(\)](#) ,
[IRCXIOAPI.IRCXIOLabel.CreateWatcher\(IRCXIOAPI.RCXIODataChangedCallback\)](#)

Interface IRCXIOAPI.IRCXMemoryIOLabel<T>

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

I/O label for memory I/O.

```
public interface IRCXIOAPI.IRCXMemoryIOLabel<T> : IRCXIOAPI.IRCXIOLabel<T>,
    IRCXIOAPI.IRCXIOLabel
```

Type Parameters

T

Data type (bool / byte / ushort).

Inherited Members

[IRCXIOAPI.IRCXIOLabel<T>.Read\(\)](#) , [IRCXIOAPI.IRCXIOLabel<T>.Write\(T\)](#) , [IRCXIOAPI.IRCXIOLabel.Label](#) ,
[IRCXIOAPI.IRCXIOLabel.Description](#) , [IRCXIOAPI.IRCXIOLabel.Port](#) ,
[IRCXIOAPI.IRCXIOLabel.GetDataTypeName\(\)](#) , [IRCXIOAPI.IRCXIOLabel.GetKind\(\)](#) ,
[IRCXIOAPI.IRCXIOLabel.CreateWatcher\(IRCXIOAPI.RCXIODataChangedCallback\)](#)

Interface IRCXIOAPI.IRCXOutputIOLabel<T>

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

I/O label for output ports.

```
public interface IRCXIOAPI.IRCXOutputIOLabel<T> : IRCXIOAPI.IRCXIOLabel<T>,
    IRCXIOAPI.IRCXIOLabel
```

Type Parameters

T

Data type (bool / byte / ushort).

Inherited Members

[IRCXIOAPI.IRCXIOLabel<T>.Read\(\)](#) , [IRCXIOAPI.IRCXIOLabel<T>.Write\(T\)](#) , [IRCXIOAPI.IRCXIOLabel.Label](#) ,
[IRCXIOAPI.IRCXIOLabel.Description](#) , [IRCXIOAPI.IRCXIOLabel.Port](#) ,
[IRCXIOAPI.IRCXIOLabel.GetDataTypeName\(\)](#) , [IRCXIOAPI.IRCXIOLabel.GetKind\(\)](#) ,
[IRCXIOAPI.IRCXIOLabel.CreateWatcher\(IRCXIOAPI.RCXIODataChangedCallback\)](#)

Delegate IRCXIOAPI.RCXIODataChanged Callback

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Callback invoked when I/O data changes.

```
public delegate void IRCXIOAPI.RCXIODataChangedCallback(IRCXIOAPI.IRCXIOWatcher watcher, int  
oldData, int newData)
```

Parameters

watcher [IRCXIOAPI.IRCXIOWatcher](#)

The watcher instance that detected the change.

oldData [int](#)

The data value before the update.

newData [int](#)

The data value after the update.

Enum IRCXIOAPI.RCXIOKind

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

The I/O kind.

```
public enum IRCXIOAPI.RCXIOKind
```

Fields

Input = 1

Input I/O.

Memory = 3

Memory I/O.

Output = 2

Output I/O.

Undefined = 0

Undefined.

Interface IRCXLangRxCaption

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Language reactive caption

```
public interface IRCXLangRxCaption : INotifyPropertyChanged
```

Inherited Members

[INotifyPropertyChanged.PropertyChanged](#) 

Properties

Value

Gets the caption string

```
string Value { get; }
```

Property Value

[string](#) 

Interface IRCXMain

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

RC+ Extensions Main

```
public interface IRCXMain
```

Properties

CaptionFilePath

Gets the file path to the caption data.

```
string CaptionFilePath { get; }
```

Property Value

[string](#)[↗]

The caption data file path.

Icon

Gets or sets the icon image.

```
ImageSource? Icon { get; set; }
```

Property Value

[ImageSource](#)[↗]

An [ImageSource](#)[↗] representing the icon, or `null` if no icon is set.

Id

Gets the Extension identifier.

```
string Id { get; }
```

Property Value

[string](#)

The unique identifier of the Extension.

Methods

GetDefaultDialogTitle()

Retrieves the dialog title.

```
string GetDefaultDialogTitle()
```

Returns

[string](#)

The dialog title string.

Remarks

Used for message boxes and similar dialogs.

Initialize(IServiceProvider, IRCXCaptionGetter, IRCXConfiguration)

Performs initialization.

```
bool Initialize(IServiceProvider apiProvider, IRCXCaptionGetter captions,  
IRCXConfiguration settings)
```


Parameters

`apiProvider` [IServiceProvider](#)

API container.

`captions` [IRCXCaptionGetter](#)

Interface for retrieving captions.

`settings` [IRCXConfiguration](#)

Interface for accessing configuration data.

Returns

[bool](#)

`true` if successful; otherwise, `false`.

Terminate()

Performs termination.

```
void Terminate()
```

Interface IRCXMainMenuItemProvider

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Provides main menu items.

```
public interface IRCXMainMenuItemProvider
```

Properties

Id

Gets the Extension identifier.

```
string Id { get; }
```

Property Value

[string](#)

MainMenuRootItem

Gets the root menu item that appears under the specified top-level menu.

```
IRCXMainMenuItemProvider.MenuItem MainMenuRootItem { get; }
```

Property Value

[IRCXMainMenuItemProvider.MenuItem](#)

The root item of the user-defined menu structure.

MenuItemId

Gets the menu item identifier.

```
string MenuItemId { get; }
```

Property Value

[string](#)

The menu item ID.

Remarks

This identifier must be unique across main menu items.

TopLevel

Gets the top-level menu.

```
IRCXMainMenuItemProvider.TopLevelMenu TopLevel { get; }
```

Property Value

[IRCXMainMenuItemProvider.TopLevelMenu](#)

The top-level (root) menu in the menu hierarchy.

Methods

ExecuteMainMenuItemCommandAsync(string, bool)

Asynchronously executes a command.

```
Task ExecuteMainMenuItemCommandAsync(string commandName, bool isFromToolBar)
```

Parameters

commandName [string](#)

The name of the command to execute.

`isFromToolBar` [bool](#)

`true` if invoked from the toolbar; otherwise, `false`.

Returns

[Task](#)

A task that represents the asynchronous operation.

Class IRCXMainMenuItemProvider.MenuItem

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents a menu item.

```
public class IRCXMainMenuItemProvider.MenuItem
```

Inheritance

[object](#)  ← IRCXMainMenuItemProvider.MenuItem

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Properties

Caption

Gets or sets the caption displayed for the menu item.

```
public RCXCaption Caption { get; set; }
```

Property Value

[RCXCaption](#)

The caption.

Children

Gets or sets the child menu items (for hierarchical menus).

```
public IList<IRCXMainMenuItemProvider.MenuItem>? Children { get; set; }
```

Property Value

[IList](#) <[IRCXMainMenuItemProvider.MenuItem](#)>

A list of child [IRCXMainMenuItemProvider.MenuItem](#) instances, or `null` if none.

CommandName

Gets or sets the command name associated with the menu item.

```
public string CommandName { get; set; }
```

Property Value

[string](#)

The command name.

Icon

Gets or sets the icon image.

```
public ImageSource? Icon { get; set; }
```

Property Value

[ImageSource](#)

An [ImageSource](#) representing the icon, or `null` if no icon is set.

ToolTip

Gets or sets the tooltip caption.

```
public RCXCaption? ToolTip { get; set; }
```

Property Value

[RCXCaption](#)

The caption to display in the tooltip; otherwise, `null` if no caption is set.

Remarks

The toolbar item is shown only when a caption is present. However, this is ignored if hierarchical menu items exist.

Enum IRCXMainMenuItemProvider.TopLevelMenu

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Specifies the top-level menu.

```
public enum IRCXMainMenuItemProvider.TopLevelMenu
```

Fields

Default = 0

The default menu (Tools).

Edit = 2

The Edit menu.

Extension = 8

The Extension menu.

File = 1

The File menu.

Help = 10

The Help menu.

Project = 4

The Project menu.

Run = 5

The Run menu.

Setup = 7

The Setup menu.

Tools = 6

The Tools menu.

View = 3

The View menu.

Window = 9

The Window menu.

Interface IRCXPointAPI

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

API related to points.

```
public interface IRCXPointAPI : INotifyPropertyChanged
```

Inherited Members

[INotifyPropertyChanged.PropertyChanged](#) 

Properties

ControllerCurrentPointFile

Gets the controller's current points file.

```
string? ControllerCurrentPointFile { get; }
```

Property Value

[string](#) 

The file name of the controller's current points file, or **null** if not set.

PointFileDescriptors

Gets the list of point file descriptors.

```
IEnumerable<IRCXPointAPI.RCXPointFileDescriptor> PointFileDescriptors { get; }
```

Property Value

[IEnumerable](#)  [<IRCXPointAPI.RCXPointFileDescriptor>](#)

A list of [IRCXPointAPI.RCXPointFileDescriptor](#) instances describing available points files.

PointNumberMax

Gets the maximum allowed point number.

```
int PointNumberMax { get; }
```

Property Value

[int](#)

The maximum point number.

PointNumberMin

Gets the minimum allowed point number.

```
int PointNumberMin { get; }
```

Property Value

[int](#)

The minimum point number.

Methods

AddPoint(string, IDictionary<string, RCXPointElement>, bool)

Adds a point to the specified file.

```
RCXCommon.RCXResult AddPoint(string fileName, IDictionary<string,  
IRCXPointAPI.RCXPointElement> point, bool shouldSave = false)
```

Parameters

fileName [string](#)

The file name.

point [IDictionary](#) <[string](#), [IRCXPointAPI.RCXPointElement](#)>

The point data.

shouldSave [bool](#)

Whether to save the point file.

Returns

[RCXCommon.RCXResult](#)

The result code.

Remarks

If a predefined point number is specified within the provided data, the existing point will be updated.

DeletePoint(string, int, bool)

Deletes a point from the specified file.

```
RCXCommon.RCXResult DeletePoint(string fileName, int pointNumber, bool shouldSave = false)
```

Parameters

fileName [string](#)

The file name.

pointNumber [int](#)

The point number.

shouldSave [bool](#)

Whether to save the point file.

Returns

[RCXCommon.RCXResult](#)

The result code.

GetPoints(string)

Gets points from the specified file.

```
(RCXCommon.RCXResult, IReadOnlyList<IDictionary<string, IRCXPointAPI.RCXPointElement>>?)  
GetPoints(string fileName)
```

Parameters

fileName [string](#)

The file name.

Returns

([RCXCommon.RCXResult](#), [IReadOnlyList](#) <[IDictionary](#) <[string](#), [IRCXPointAPI.RCXPointElement](#)>>)

A tuple containing the [RCXCommon.RCXResult](#) and the points data.

SetControllerCurrentPointFileAsync(string)

Sets the controller's current points file asynchronously.

```
Task<RCXCommon.RCXResult> SetControllerCurrentPointFileAsync(string fileName)
```

Parameters

fileName [string](#)

The file name.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

TeachPoint(string, int, string?, string?, bool, bool)

Sets the current position of the current robot as a point.

```
RCXCommon.RCXResult TeachPoint(string fileName, int pointNumber, string? label = null,  
string? description = null, bool simVisible = false, bool shouldSave = false)
```

Parameters

fileName [string](#)

The file name.

pointNumber [int](#)

The point number.

label [string](#)

The label to assign; specify `null` to omit.

description [string](#)

The description/comment to assign; specify `null` to omit.

simVisible [bool](#)

Whether the point is visible in the simulator.

shouldSave [bool](#)

Whether to save the point file.

Returns

[RCXCommon.RCXResult](#)

The result code.

Events

PointsChanged

Occurs when points have been changed.

event IRCXPointAPI.PointsChangedEventHandler PointsChanged

Event Type

[IRCXPointAPI.PointsChangedEventHandler](#)

Class IRCXPointAPI.PointsChangedEventArgs

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Provides event data for a points change.

```
public class IRCXPointAPI.PointsChangedEventArgs : EventArgs
```

Inheritance

[object](#)  ← [EventArgs](#)  ← IRCXPointAPI.PointsChangedEventArgs

Inherited Members

[EventArgs.Empty](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Constructors

PointsChangedEventArgs(string)

Initializes a new instance of the [IRCXPointAPI.PointsChangedEventArgs](#) class.

```
public PointsChangedEventArgs(string fileName)
```

Parameters

fileName [string](#) 

The file name.

Properties

FileName

Gets the points file name.


```
public string FileName { get; }
```

Property Value

[string](#) 

The file name associated with the points change.

Delegate IRCXPointAPI.PointsChangedEventHandler

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents the delegate for handling points changed events.

```
public delegate void IRCXPointAPI.PointsChangedEventHandler(object sender,  
IRCXPointAPI.PointsChangedEventArgs ev)
```

Parameters

sender [object](#)

The source of the event.

ev [IRCXPointAPI.PointsChangedEventArgs](#)

The event data.

Class IRCXPointAPI.RCXPointElement

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents a point element value.

```
public class IRCXPointAPI.RCXPointElement
```

Inheritance

[object](#)  ← IRCXPointAPI.RCXPointElement

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

RCXPointElement(Type, object)

Initializes a new instance of the [IRCXPointAPI.RCXPointElement](#) class.

```
public RCXPointElement(Type valueType, object value)
```

Parameters

valueType [Type](#) 

The type of the value.

value [object](#) 

The value.

Properties

Value

Gets or sets the value.

```
public object Value { get; set; }
```

Property Value

[object](#) 

The point element value.

ValueType

Gets the type of the value.

```
public Type ValueType { get; }
```

Property Value

[Type](#) 

The runtime type of the value.

Enum IRCXPointAPI.RCXPointElement.Elbow

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Specifies the elbow configuration.

```
public enum IRCXPointAPI.RCXPointElement.Elbow
```

Fields

Above = 1

Below = 2

Enum IRCXPointAPI.RCXPointElement.Hand

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Specifies the hand orientation.

```
public enum IRCXPointAPI.RCXPointElement.Hand
```

Fields

Lefty = 2

Righty = 1

Enum IRCXPointAPI.RCXPointElement.Wrist

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Specifies the wrist orientation.

```
public enum IRCXPointAPI.RCXPointElement.Wrist
```

Fields

```
Flip = 2
```

```
NotFlip = 1
```

Class IRCXPointAPI.RCXPointFileDescriptor

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Descriptor for a points file.

```
public class IRCXPointAPI.RCXPointFileDescriptor
```

Inheritance

[object](#) ← IRCXPointAPI.RCXPointFileDescriptor

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

RCXPointFileDescriptor(string, int?, bool)

Initializes a new instance of the [IRCXPointAPI.RCXPointFileDescriptor](#) class.

```
public RCXPointFileDescriptor(string fileName, int? robotNumber, bool isDefault)
```

Parameters

fileName [string](#)

The points file name.

robotNumber [int](#)?

The associated robot number; specify **null** for a common/shared file.

isDefault [bool](#)

A value indicating whether this is the default file.

Properties

FileName

Gets the file name of the points file.

```
public string FileName { get; }
```

Property Value

[string](#)

The points file name.

IsDefault

Gets a value indicating whether this is the default points file.

```
public bool IsDefault { get; }
```

Property Value

[bool](#)

`true` if this is the default; otherwise, `false`.

RobotNumber

Gets the associated robot number; `null` indicates a common/shared file.

```
public int? RobotNumber { get; }
```

Property Value

[int](#)?

The robot number linked to this points file, or `null` if the file is common/shared.

Interface IRCXPreferencesAPI

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

API related to Preferences.

```
public interface IRCXPreferencesAPI
```

Methods

GetPreferences(string)

Gets the Preferences for the specified category.

```
(RCXCommon.RCXResult, IDictionary<string, IRCXPreferencesAPI.PreferenceValue>?)  
GetPreferences(string categoryName)
```

Parameters

categoryName [string](#)

The Preferences category name.

Returns

[\(RCXCommon.RCXResult, IDictionary<string, IRCXPreferencesAPI.PreferenceValue>\)](#)

A tuple consisting of the result code and a dictionary of preference values.

GetPreferencesCategoryNames()

Gets the list of category names for Preferences.

```
IEnumerable<string> GetPreferencesCategoryNames()
```

Returns

[IEnumerable](#) <[string](#)>

A collection of category names for Preferences.

SetPreferencesAsync(string, IDictionary<string, PreferenceValue>)

Sets (updates) the Preferences for the specified category asynchronously.

```
Task<RCXCommon.RCXResult> SetPreferencesAsync(string categoryName, IDictionary<string,  
IRCXPreferencesAPI.PreferenceValue> preferences)
```

Parameters

categoryName [string](#)

The Preferences category name.

preferences [IDictionary](#) <[string](#), [IRCXPreferencesAPI.PreferenceValue](#)>

A dictionary of preference values to apply.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

Class IRCXPreferencesAPI.PreferenceValue

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents a preference value for Preferences.

```
public class IRCXPreferencesAPI.PreferenceValue : RCXCommon.RCXSettingValue
```

Inheritance

[object](#)  ← [RCXCommon.RCXSettingValue](#) ← IRCXPreferencesAPI.PreferenceValue

Inherited Members

[RCXCommon.RCXSettingValue.Value](#) , [RCXCommon.RCXSettingValue.ValueType](#) ,
[RCXCommon.RCXSettingValue.CanSet](#) , [RCXCommon.RCXSettingValue.Choices](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Interface IRCXProgramEditorAPI

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

API related to the Program Editor.

```
public interface IRCXProgramEditorAPI : INotifyPropertyChanged
```

Inherited Members

[INotifyPropertyChanged.PropertyChanged](#) 

Properties

ProgramEditors

Gets the list of Program Editors that are currently open.

```
IEnumerable<IRCXProgramEditorAPI.IRCXProgramEditor>? ProgramEditors { get; }
```

Property Value

[IEnumerable](#)  <[IRCXProgramEditorAPI.IRCXProgramEditor](#)>

A collection of open [IRCXProgramEditorAPI.IRCXProgramEditor](#) instances; **null** if an error occurs.

Interface IRCXProgramEditorAPI.IRCXProgramEditor

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents the Program Editor.

```
public interface IRCXProgramEditorAPI.IRCXProgramEditor : INotifyPropertyChanged
```

Inherited Members

[INotifyPropertyChanged.PropertyChanged](#) 

Properties

Breakpoints

Gets the set of breakpoint lines.

```
HashSet<int> Breakpoints { get; }
```

Property Value

[HashSet](#)  <[int](#)  >

A set of line numbers where breakpoints are defined.

FileName

Gets the file name.

```
string FileName { get; }
```

Property Value

[string](#) 

The name of the file.

IsDataLoaded

Gets whether the data has been loaded.

```
bool IsDataLoaded { get; }
```

Property Value

[bool](#)

true if the data is loaded; otherwise, **false**.

Remarks

When **false**, APIs other than **FileName** and **IsDataLoaded** are unavailable (because data is not loaded).

LineCount

Gets the number of lines.

```
int LineCount { get; }
```

Property Value

[int](#)

The number of lines.

Methods

CanPlaceBreakpointAsync(int)

Asynchronously determines whether a breakpoint can be placed on the specified line.

```
Task<(RCXCommon.RCXResult, bool)> CanPlaceBreakpointAsync(int lineNumber)
```

Parameters

lineNumber [int](#)

The line number (starting at 1).

Returns

[Task](#) <[RCXCommon.RCXResult](#), [bool](#)>

A task that returns a tuple consisting of the result code and a boolean value indicating whether a breakpoint can be placed.

ClearAllBreakpointsAsync()

Asynchronously clears all breakpoints in the file.

```
Task<RCXCommon.RCXResult> ClearAllBreakpointsAsync()
```

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

ClearBreakpointAsync(int)

Asynchronously clears the breakpoint on the specified line.

```
Task<RCXCommon.RCXResult> ClearBreakpointAsync(int lineNumber)
```

Parameters

lineNumber [int](#)

The line number (starting at 1).

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

GetLineContentAsync(int)

Asynchronously gets the content of a line.

```
Task<(RCXCommon.RCXResult, string?)> GetLineContentAsync(int lineNumber)
```

Parameters

lineNumber [int](#)

The line number (starting at 1).

Returns

[Task](#) <[RCXCommon.RCXResult](#), [string](#)>

A task that returns a tuple consisting of the result code and the line content. The line content is `null` if the line does not exist or cannot be retrieved.

SetBreakpointAsync(int)

Asynchronously sets a breakpoint on the specified line.

```
Task<RCXCommon.RCXResult> SetBreakpointAsync(int lineNumber)
```

Parameters

lineNumber [int](#)

The line number (starting at 1).

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

ToggleBreakpointAsync(int)

Asynchronously toggles the presence of a breakpoint on the specified line.

```
Task<RCXCommon.RCXResult> ToggleBreakpointAsync(int lineNumber)
```

Parameters

lineNumber [int](#)

The line number (starting at 1).

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

Interface IRCXProgramExecutionAPI

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

API related to program execution.

```
public interface IRCXProgramExecutionAPI : INotifyPropertyChanged
```

Inherited Members

[INotifyPropertyChanged.PropertyChanged](#) 

Properties

ForceLowPower

Gets a value indicating whether low-power mode is forced.

```
bool? ForceLowPower { get; }
```

Property Value

[bool](#) ?

true if low-power mode is forced; otherwise, **false**. **null** if unspecified.

SpeedFactor

Gets the speed factor (ratio) for program execution.

```
int? SpeedFactor { get; }
```

Property Value

[int](#) ?

An integer representing the speed factor; **null** if unspecified.

Tasks

Gets the list of tasks.

```
IEnumerable<IRCXProgramExecutionAPI.IRCXTask> Tasks { get; }
```

Property Value

[IEnumerable](#) <[IRCXProgramExecutionAPI.IRCXTask](#)>

UserForms

Gets the list of user form names.

```
IEnumerable<string>? UserForms { get; }
```

Property Value

[IEnumerable](#) <[string](#)>

A collection of user form names; **null** if unavailable due to no open project or an error.

UserFunctions

Gets the list of user function names.

```
IEnumerable<string>? UserFunctions { get; }
```

Property Value

[IEnumerable](#) <[string](#)>

A collection of user function names; **null** if unavailable due to no open project or an error.

Methods

ContinueAsync(string)

Resumes program execution from a paused state.

```
Task<RCXCommon.RCXResult> ContinueAsync(string extensionId)
```

Parameters

extensionId [string](#)

The Extension ID.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code.

EnablePeriodicTaskStateWatchingAsync(bool)

Starts or stops periodic monitoring of task states asynchronously.

```
Task<RCXCommon.RCXResult> EnablePeriodicTaskStateWatchingAsync(bool enable)
```

Parameters

enable [bool](#)

Whether periodic task state monitoring is enabled (true to start, false to stop).

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

ExecuteSpelCommandAsync(string)

Asynchronously executes a SPEL+ command.

```
Task<(RCXCommon.RCXResult, int, string)> ExecuteSpellCommandAsync(string command)
```

Parameters

command [string](#)

The command line to execute.

Returns

[Task](#) <(RCXCommon.RCXResult, [int](#), [string](#))>

A task that returns a tuple consisting of the result code, the error code, and the output string.

GetFunctionInfo(int)

Gets the function information.

```
(RCXCommon.RCXResult, IRCXProgramExecutionAPI.IRCXFunctionInfo?)  
GetFunctionInfo(int functionId)
```

Parameters

functionId [int](#)

The function ID.

Returns

(RCXCommon.RCXResult, IRCXProgramExecutionAPI.IRCXFunctionInfo)

A tuple containing the result code and the function information (or null if not found).

GetGlobalVariables()

Gets the list of global variables.

```
(RCXCommon.RCXResult, IEnumerable<IRCXProgramExecutionAPI.IRCXGlobalVariable>?)
```

GetGlobalVariables()

Returns

([RCXCommon.RCXResult](#), [IEnumerable](#) <[IRCXProgramExecutionAPI.IRCXGlobalVariable](#)>)

A tuple containing the result code and the list of global variables (or [null](#) if unavailable).

GetLocalVariables(int, IRCXCallStackEntry)

Gets the list of local variables.

```
(RCXCommon.RCXResult, IEnumerable<IRCXProgramExecutionAPI.IRCXLocalVariable>?)  
GetLocalVariables(int taskNumber, IRCXProgramExecutionAPI.IRCXTask.IRCXCallStackEntry  
callStackEntry)
```

Parameters

taskNumber [int](#)

The task number.

callStackEntry [IRCXProgramExecutionAPI.IRCXTask.IRCXCallStackEntry](#)

The call stack entry.

Returns

([RCXCommon.RCXResult](#), [IEnumerable](#) <[IRCXProgramExecutionAPI.IRCXLocalVariable](#)>)

A tuple containing the result code and the list of local variables (or [null](#) if unavailable).

GetModuleVariables()

Gets the list of module variables.

```
(RCXCommon.RCXResult, IEnumerable<IRCXProgramExecutionAPI.IRCXModuleVariable>?)  
GetModuleVariables()
```

Returns

([RCXCommon.RCXResult](#), [IEnumerable](#) <[IRCXProgramExecutionAPI.IRCXModuleVariable](#)>)

A tuple containing the result code and the list of module variables (or [null](#) if unavailable).

PauseAsync(string)

Pauses program execution.

```
Task<RCXCommon.RCXResult> PauseAsync(string extensionId)
```

Parameters

extensionId [string](#)

The Extension ID.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code.

SetForceLowPowerAsync(bool)

Asynchronously sets whether low-power mode is forced.

```
Task<RCXCommon.RCXResult> SetForceLowPowerAsync(bool forceLowPower)
```

Parameters

forceLowPower [bool](#)

The setting value indicating whether to force low-power mode.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code.

SetSpeedFactorAsync(int)

Asynchronously sets the speed factor (ratio) for program execution.

```
Task<RCXCommon.RCXResult> SetSpeedFactorAsync(int speedFactor)
```

Parameters

speedFactor [int](#)

The speed factor to apply.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code.

StartFormAsync(string, bool, string)

Starts program execution for the specified form.

```
Task<RCXCommon.RCXResult> StartFormAsync(string formName, bool withBGMain,  
string extensionId)
```

Parameters

formName [string](#)

The form name.

withBGMain [bool](#)

Whether to also start BGMain.

extensionId [string](#)

The Extension ID.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code.

StartFunctionAsync(string, bool, bool, string)

Starts program execution for the specified function.

```
Task<RCXCommon.RCXResult> StartFunctionAsync(string functionName, bool asBackgroundTask,  
bool withBGMain, string extensionId)
```

Parameters

functionName [string](#)

The function name.

asBackgroundTask [bool](#)

Whether to run as a background task.

withBGMain [bool](#)

Whether to also start BGMain.

extensionId [string](#)

The Extension ID.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code.

Remarks

When **asBackgroundTask** is **true**, **withBGMain** is ignored.

StopAsync(bool, string)

Stops program execution.

```
Task<RCXCommon.RCXResult> StopAsync(bool normalTasksOnly, string extensionId)
```

Parameters

normalTasksOnly [bool](#)

Whether to target only normal tasks.

extensionId [string](#)

The Extension ID.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code.

Interface IRCXProgramExecutionAPI. IRCXFunctionInfo

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Function information.

```
public interface IRCXProgramExecutionAPI.IRCXFunctionInfo
```

Properties

FileName

Gets the file name where the function is defined.

```
string FileName { get; }
```

Property Value

[string](#)

LineNumber

Gets the line number.

```
int LineNumber { get; }
```

Property Value

[int](#)

Name

Gets the function name.

```
string Name { get; }
```

Property Value

[string](#) 

Interface IRCXProgramExecutionAPI. IRCXGlobalVariable

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Global variable object.

```
public interface IRCXProgramExecutionAPI.IRCXGlobalVariable :  
    IRCXProgramExecutionAPI.IRCXVariable
```

Inherited Members

[IRCXProgramExecutionAPI.IRCXVariable.Name](#) , [IRCXProgramExecutionAPI.IRCXVariable.SPELType](#) ,
[IRCXProgramExecutionAPI.IRCXVariable.CSType](#) , [IRCXProgramExecutionAPI.IRCXVariable.DimSize](#) ,
[IRCXProgramExecutionAPI.IRCXVariable.GetValue<T>\(out T, int\[\]\)](#) ,
[IRCXProgramExecutionAPI.IRCXVariable.SetValue<T>\(T, int\[\]\)](#).

Properties

IsPreserved

Gets whether the variable is a preserved variable.

```
bool IsPreserved { get; }
```

Property Value

[bool](#)

[true](#) if the variable is a preserved variable; otherwise, [false](#).

Interface IRCXProgramExecutionAPI.IRCXLocalVariable

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Local variable object.

```
public interface IRCXProgramExecutionAPI.IRCXLocalVariable :  
    IRCXProgramExecutionAPI.IRCXVariable
```

Inherited Members

[IRCXProgramExecutionAPI.IRCXVariable.Name](#) , [IRCXProgramExecutionAPI.IRCXVariable.SPELType](#) ,
[IRCXProgramExecutionAPI.IRCXVariable.CSType](#) , [IRCXProgramExecutionAPI.IRCXVariable.DimSize](#) ,
[IRCXProgramExecutionAPI.IRCXVariable.GetValue<T>\(out T, int\[\]\)](#) ,
[IRCXProgramExecutionAPI.IRCXVariable.SetValue<T>\(T, int\[\]\)](#).

Properties

FunctionId

Gets the function ID.

```
int FunctionId { get; }
```

Property Value

[int](#)

The ID of the function to which this local variable belongs.

Position

Gets the position from the top of the call stack.

```
int Position { get; }
```

Property Value

[int](#)

The position from the top of the call stack for this local variable.

TaskNumber

Gets the task number.

```
int TaskNumber { get; }
```

Property Value

[int](#)

The number identifying the task to which this local variable belongs.

Interface IRCXProgramExecutionAPI. IRCXModuleVariable

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Module variable object.

```
public interface IRCXProgramExecutionAPI.IRCXModuleVariable :  
    IRCXProgramExecutionAPI.IRCXVariable
```

Inherited Members

[IRCXProgramExecutionAPI.IRCXVariable.Name](#) , [IRCXProgramExecutionAPI.IRCXVariable.SPELType](#) ,
[IRCXProgramExecutionAPI.IRCXVariable.CSType](#) , [IRCXProgramExecutionAPI.IRCXVariable.DimSize](#) ,
[IRCXProgramExecutionAPI.IRCXVariable.GetValue<T>\(out T, int\[\]\)](#) ,
[IRCXProgramExecutionAPI.IRCXVariable.SetValue<T>\(T, int\[\]\)](#).

Properties

FileName

Gets the file name.

```
string FileName { get; }
```

Property Value

[string](#) 

The file name in which the module variable is defined.

Interface IRCXProgramExecutionAPI.IRCXTask

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents a task object.

```
public interface IRCXProgramExecutionAPI.IRCXTask
```

Properties

CPU

Gets the CPU load percentage.

```
int? CPU { get; }
```

Property Value

[int](#).

The CPU load percentage; **null** if unavailable.

ErrorNumber

Gets the error number.

```
int ErrorNumber { get; }
```

Property Value

[int](#).

The error number.

FinishTime

Gets the finish time.

```
DateTime? FinishTime { get; }
```

Property Value

[DateTime](#)[↗]?

The finish time; **null** if not finished or unknown.

Remarks

If connected to an RC700 controller, this property always returns null.

FunctionId

Gets the function ID.

```
int FunctionId { get; }
```

Property Value

[int](#)[↗]

The function ID.

JointNumber

Gets the joint number.

```
int JointNumber { get; }
```

Property Value

[int](#)[↗]

The joint number.

Kind

Gets the task kind.

```
IRCXProgramExecutionAPI.IRCXTask.RCXTaskKind Kind { get; }
```

Property Value

[IRCXProgramExecutionAPI.IRCXTask.RCXTaskKind](#)

The task kind.

LineNumber

Gets the line number.

```
int LineNumber { get; }
```

Property Value

[int](#)

The line number.

Name

Gets the task name.

```
string Name { get; }
```

Property Value

[string](#)

The task name.

Number

Gets the task number.

```
int Number { get; }
```

Property Value

[int](#)

The task number.

RobotNumber

Gets the robot number.

```
int RobotNumber { get; }
```

Property Value

[int](#)

The robot number.

StartTime

Gets the start time.

```
DateTime? StartTime { get; }
```

Property Value

[DateTime](#)?

The start time; `null` if not started or unknown.

State

Gets the current task state.

```
IRCXProgramExecutionAPI.IRCXTask.RCXTaskState State { get; }
```

Property Value

[IRCXProgramExecutionAPI.IRCXTask.RCXTaskState](#)

The task state.

Methods

GetCallStack()

Gets the call stack.

```
IList<IRCXProgramExecutionAPI.IRCXTask.IRCXCallStackEntry>? GetCallStack()
```

Returns

[IList](#) <[IRCXProgramExecutionAPI.IRCXTask.IRCXCallStackEntry](#)>

The call stack as a list of [IRCXProgramExecutionAPI.IRCXTask.IRCXCallStackEntry](#); **null** if an error occurs.

HaltAsync()

Pauses (halts) the task asynchronously.

```
Task<RCXCommon.RCXResult> HaltAsync()
```

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code.

Remarks

Tasks marked as **NoPause** or **NoEmgAbort** are also paused.

ResumeAsync()

Resumes a task that is in the Halt state asynchronously.

```
Task<RCXCommon.RCXResult> ResumeAsync()
```

Returns

[Task](#)  <[RCXCommon.RCXResult](#)>

A task that returns the result code.

StepInAsync()

Steps into asynchronously.

```
Task<RCXCommon.RCXResult> StepInAsync()
```

Returns

[Task](#)  <[RCXCommon.RCXResult](#)>

A task that returns the result code.

StepOverAsync()

Steps over asynchronously.

```
Task<RCXCommon.RCXResult> StepOverAsync()
```

Returns

[Task](#)  <[RCXCommon.RCXResult](#)>

A task that returns the result code.

WalkAsync()

Walk asynchronously.

```
Task<RCXCommon.RCXResult> WalkAsync()
```

Returns

[Task](#)  <[RCXCommon.RCXResult](#)>

A task that returns the result code.

Remarks

Runs until the next action or output instruction is encountered, then stops at the following line.

Interface IRCXProgramExecutionAPI.IRCXTask.IRCXCallStackEntry

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents a call stack entry.

```
public interface IRCXProgramExecutionAPI.IRCXTask.IRCXCallStackEntry
```

Properties

FunctionId

Gets the function ID.

```
int FunctionId { get; }
```

Property Value

[int](#)

The unique identifier of the function.

LineNumber

Gets the line number.

```
int LineNumber { get; }
```

Property Value

[int](#)

The line number.

Position

Gets the index counted from the top of the call stack.

```
int Position { get; }
```

Property Value

[int](#)

The index from the top. The smallest index corresponds to the top entry.

Enum IRCXProgramExecutionAPI.IRCXTask.RCXTaskKind

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents the kind of a task.

```
public enum IRCXProgramExecutionAPI.IRCXTask.RCXTaskKind
```

Fields

Background = 2

Normal = 1

Other = 0

Trap = 3

Enum IRCXProgramExecutionAPI.IRCXTask.RCXTaskState

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents the state of a task.

```
public enum IRCXProgramExecutionAPI.IRCXTask.RCXTaskState
```

Fields

Aborted = 10

Break = 3

Error = 8

Finished = 9

Halt = 6

Other = 0

Pause = 7

Run = 1

Step = 4

Wait = 2

Walk = 5

Interface IRCXProgramExecutionAPI. IRCXVariable

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Variable object.

```
public interface IRCXProgramExecutionAPI.IRCXVariable
```

Properties

CSType

Gets the value type in C#.

```
Type CSType { get; }
```

Property Value

[Type](#)

The CLR [Type](#) corresponding to the variable's value in C#.

DimSize

Gets the maximum element count per dimension.

```
int[] DimSize { get; }
```

Property Value

[int](#) []

An array of sizes for each dimension. For non-array variables, this is empty.

Name

Gets the variable name.

```
string Name { get; }
```

Property Value

[string](#)

The name of the variable.

SPELType

Gets the type name in SPEL+.

```
string SPELType { get; }
```

Property Value

[string](#)

The SPEL+ type name used to represent this variable.

Methods

GetValue<T>(out T, int[]?)

Gets the value.

```
RCXCommon.RCXResult GetValue<T>(out T value, int[]? indices = null)
```

Parameters

value T

The variable that returns the value.

indices [int\[\]](#)

Indices when the variable is an array.

Returns

[RCXCommon.RCXResult](#)

The result code.

Type Parameters

T

The type.

Remarks

The **indices** parameter is ignored for non-array variables.

SetValue<T>(T, int[]?)

Sets the value.

```
RCXCommon.RCXResult SetValue<T>(T value, int[]? indices = null)
```

Parameters

value T

The value to set.

indices [int\[\]](#)

Indices when the variable is an array.

Returns

[RCXCommon.RCXResult](#)

The result code.

Type Parameters

T

The type.

Remarks

The **indices** parameter is ignored for non-array variables.

Interface IRCXProjectAPI

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Project-related API.

```
public interface IRCXProjectAPI : INotifyPropertyChanged
```

Inherited Members

[INotifyPropertyChanged.PropertyChanged](#) 

Properties

ProjectFiles

Gets the list of files in the currently open project.

```
IEnumerable<IRCXProjectAPI.ProjectFileInfo>? ProjectFiles { get; }
```

Property Value

[IEnumerable](#)  <[IRCXProjectAPI.ProjectFileInfo](#)>

A list of [IRCXProjectAPI.ProjectFileInfo](#) items for the currently open project, or [null](#)  if no project is open.

ProjectFolder

Gets the project folder path name, or [null](#)  if no project is open.

```
string? ProjectFolder { get; }
```

Property Value

[string](#) 

The full path of the currently opened project folder, or [null](#) if no project is open.

RecentlyUsedProjectFolders

Gets the list of recently used project folder path names.

```
IEnumerable<string> RecentlyUsedProjectFolders { get; }
```

Property Value

[IEnumerable](#) <[string](#)>

A collection of full paths for recently used project folders.

Methods

AddFileToProjectAsync(string, string)

Adds an existing file to the currently open project asynchronously.

```
Task AddFileToProjectAsync(string filter, string extensionId)
```

Parameters

filter [string](#)

The filter string for the file selection dialog.

extensionId [string](#)

The Extension ID.

Returns

[Task](#)

A task that represents the asynchronous operation.

Remarks

Displays a file selection dialog.

AddSpecificFileToProjectAsync(string, string)

Adds the specified existing file to the currently open project asynchronously.

```
Task AddSpecificFileToProjectAsync(string filePath, string extensionId)
```

Parameters

filePath [string](#)

The full file path.

extensionId [string](#)

The Extension ID.

Returns

[Task](#)

A task that represents the asynchronous operation.

BuildAsync(bool)

Builds the project asynchronously.

```
Task<RCXCommon.RCXResult> BuildAsync(bool rebuild = false)
```

Parameters

rebuild [bool](#)

Whether to perform a rebuild.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

CloseAsync()

Closes the project asynchronously.

```
Task<RCXCommon.RCXResult> CloseAsync()
```

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

Remarks

If a save operation is canceled, [Canceled](#) is returned.

CloseProjectFileAsync(string)

Closes an open file in the currently open project asynchronously.

```
Task<RCXCommon.RCXResult> CloseProjectFileAsync(string fileName)
```

Parameters

fileName [string](#)

The file name.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

CopyAsync(string, string?)

Copies the currently open project asynchronously.

```
Task<RCXCommon.RCXResult> CopyAsync(string destinationFolder, string? projectName)
```

Parameters

destinationFolder [string](#)

The destination folder path name.

projectName [string](#)

The project name; if [null](#), the same as the currently open project.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

CreateFileAsync(string, string)

Adds a new file to the currently open project asynchronously.

```
Task<string?> CreateFileAsync(string fileExtension, string extensionId)
```

Parameters

fileExtension [string](#)

The file extension.

extensionId [string](#)

The Extension ID.

Returns

[Task](#) <[string](#)>

A task that returns the created file name, or [null](#) if the operation is canceled.

Remarks

Displays a file name input dialog.

DecryptProjectFileAsync(string, char[])

Decrypts an encrypted project file asynchronously.

```
Task<RCXCommon.RCXResult> DecryptProjectFileAsync(string fileName, char[] password)
```

Parameters

fileName [string](#) 

The file name.

password [char](#)  []

The password.

Returns

[Task](#)  <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

DeleteAsync(string)

Deletes a project asynchronously.

```
Task<RCXCommon.RCXResult> DeleteAsync(string projectFolder)
```

Parameters

projectFolder [string](#) 

The project folder path name.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

DeleteFileAsync(string, string, bool)

Deletes a file from the currently open project asynchronously.

```
Task DeleteFileAsync(string fileName, string extensionId, bool force = false)
```

Parameters

fileName [string](#)

The file name.

extensionId [string](#)

The Extension ID.

force [bool](#)

[true](#) to perform the operation without showing confirmation dialogs (i.e., suppress confirmation); otherwise, [false](#).

Returns

[Task](#)

A task that represents the asynchronous operation.

EncryptProjectFileAsync(string, char[])

Encrypts a project file asynchronously.

```
Task<RCXCommon.RCXResult> EncryptProjectFileAsync(string fileName, char[] password)
```

Parameters

fileName [string](#)

The file name.

password [char](#)[]

The password.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

GetDefaultBackupFolder()

Retrieves the path of the backup folder.

```
string GetDefaultBackupFolder()
```

Returns

[string](#)

The path of the backup folder.

GetDefaultProjectFolder()

Gets the default project folder path name based on Preferences.

```
string GetDefaultProjectFolder()
```

Returns

[string](#)

The default project folder path name derived from Preferences.

GetProjectProperties(string)

Gets the project properties for the specified category.


```
(RCXCommon.RCXResult, IDictionary<string, IRCXProjectAPI.ProjectPropertyValue>?)  
GetProjectProperties(string categoryName)
```

Parameters

categoryName [string](#)

The category name.

Returns

[\(RCXCommon.RCXResult, IDictionary<string, IRCXProjectAPI.ProjectPropertyValue>\)](#)

A tuple containing the result code and a dictionary of properties; the dictionary may be null.

GetProjectPropertiesCategoryNames()

Gets the list of project property category names.

```
IEnumerable<string> GetProjectPropertiesCategoryNames()
```

Returns

[IEnumerable<string>](#)

A list of category names.

GetProjectTemplateNameNames()

Gets the list of project template names.

```
IEnumerable<string> GetProjectTemplateNameNames()
```

Returns

[IEnumerable<string>](#)

A list of template names.

Remarks

The names depend on the current display language.

ImportFromControllerAsync(string, string?)

Imports a project from a controller.

```
Task<RCXCommon.RCXResult> ImportFromControllerAsync(string destinationFolder,  
string? projectName)
```

Parameters

destinationFolder [string](#)

The destination folder path name.

projectName [string](#)

The project name; if [null](#), the same as the source.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

ImportFromLocalAsync(string, string, string?)

Imports a project from PC or from a backup folder asynchronously.

```
Task<RCXCommon.RCXResult> ImportFromLocalAsync(string sourceFolder, string  
destinationFolder, string? projectName)
```

Parameters

sourceFolder [string](#)

The source folder path name.

destinationFolder [string](#)

The destination folder path name.

projectName [string](#)

The project name; if [null](#), the same as the source.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

NewAsync(string, string, string?)

Creates a project asynchronously.

```
Task<RCXCommon.RCXResult> NewAsync(string destinationFolder, string projectName,  
string? templateName)
```

Parameters

destinationFolder [string](#)

The destination folder path name.

projectName [string](#)

The project name.

templateName [string](#)

The template name; set [null](#) if not specified.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

Remarks

Differences from executing via the RC+ menu:

- The created project is not opened.
- No error message dialogs are shown.

OpenAsync(string, bool)

Opens a project asynchronously.

```
Task<RCXCommon.RCXResult> OpenAsync(string projectFolder, bool readOnly = false)
```

Parameters

projectFolder [string](#)

The project folder path name.

readOnly [bool](#)

Whether to open the project in read-only mode.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

OpenProjectFileAsync(string)

Opens a file in the currently open project asynchronously.

```
Task<RCXCommon.RCXResult> OpenProjectFileAsync(string fileName)
```

Parameters

fileName [string](#)

The file name.

Returns

[Task](#)  <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

ReloadProjectFileAsync(string)

Reloads an open file in the currently open project asynchronously.

```
Task<RCXCommon.RCXResult> ReloadProjectFileAsync(string fileName)
```

Parameters

fileName [string](#) 

The file name.

Returns

[Task](#)  <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

Remarks

A confirmation dialog is shown. Even if the operation is canceled, the result code is [Success](#).

RemoveFileFromProjectAsync(string, string, bool)

Excludes the specified file from the currently open project asynchronously.

```
Task RemoveFileFromProjectAsync(string fileName, string extensionId, bool force = false)
```

Parameters

fileName [string](#) 

The file name.

extensionId [string](#)

The Extension ID.

force [bool](#)

[true](#) to perform the operation without showing confirmation dialogs (i.e., suppress confirmation); otherwise, [false](#).

Returns

[Task](#)

A task that represents the asynchronous operation.

RenameAsync(string)

Renames the currently open project asynchronously.

```
Task<RCXCommon.RCXResult> RenameAsync(string newProjectName)
```

Parameters

newProjectName [string](#)

The new project name.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

RenameProjectFileAsync(string, string)

Renames a file in the currently open project asynchronously.

```
Task<RCXCommon.RCXResult> RenameProjectFileAsync(string fileName, string newFileName)
```

Parameters

fileName [string](#)

The original file name.

newFileName [string](#)

The new file name.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

SaveAsync()

Saves the project asynchronously.

```
Task<RCXCommon.RCXResult> SaveAsync()
```

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

SaveProjectFileAsync(string)

Saves an open file in the currently open project asynchronously.

```
Task<RCXCommon.RCXResult> SaveProjectFileAsync(string fileName)
```

Parameters

fileName [string](#)

The file name.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

SetProjectPropertiesAsync(string, IDictionary<string, ProjectPropertyValue>)

Sets project properties for the specified category asynchronously.

```
Task<RCXCommon.RCXResult> SetProjectPropertiesAsync(string categoryName, IDictionary<string,  
IRCXProjectAPI.ProjectPropertyValue> properties)
```

Parameters

categoryName [string](#)

The category name.

properties [IDictionary](#) <[string](#), [IRCXProjectAPI.ProjectPropertyValue](#)>

A dictionary of properties to set.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code.

SetStoreInControllerProjectFileAsync(string, bool)

Sets whether the specified project file should be stored in the controller asynchronously.

```
Task<RCXCommon.RCXResult> SetStoreInControllerProjectFileAsync(string fileName,  
bool storeInController)
```

Parameters

fileName [string](#)

The name of the file.

storeInController [bool](#)

A flag indicating whether the file should be stored in the controller.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code of the operation.

Class IRCXProjectAPI.ProjectFileInfo

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Project file information.

```
public class IRCXProjectAPI.ProjectFileInfo
```

Inheritance

[object](#) ← IRCXProjectAPI.ProjectFileInfo

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

ProjectFileInfo(string, bool, bool?, bool?)

Initializes a new instance of the [IRCXProjectAPI.ProjectFileInfo](#) class.

```
public ProjectFileInfo(string name, bool canRename, bool? storeInController = null, bool?  
encrypted = null)
```

Parameters

name [string](#)

The file name.

canRename [bool](#)

A value indicating whether the file name may be changed.

storeInController [bool](#)?

Whether the file should be stored in the controller; specify `null` if not applicable.

encrypted [bool](#)?

Whether the file is encrypted; specify `null` if not applicable.

Properties

CanRename

Gets whether the file name can be changed.

```
public bool CanRename { get; }
```

Property Value

[bool](#)[↗]

[true](#)[↗] if renaming is allowed; otherwise, [false](#)[↗].

Encrypted

Gets whether the file is encrypted.

```
public bool? Encrypted { get; }
```

Property Value

[bool](#)[↗]?

[true](#)[↗] if the file is encrypted; [false](#)[↗] if it is not; otherwise, [null](#)[↗] if not applicable.

Remarks

For files to which this attribute does not apply, [null](#)[↗] is returned.

ExtensionFileName

Gets or sets the file type name when the file belongs to an Extension.

```
public string? ExtensionFileName { get; set; }
```

Property Value

[string](#)

The Extension-specific file type name, or [null](#).

Remarks

[null](#) if the file does not belong to an Extension.

ExtensionId

Gets or sets the Extension ID.

```
public string? ExtensionId { get; set; }
```

Property Value

[string](#)

The identifier of the Extension this file belongs to, or [null](#).

Remarks

[null](#) if the file does not belong to an Extension.

Name

Gets the file name.

```
public string Name { get; }
```

Property Value

[string](#)

The name of the project file.

StoreInController

Gets whether the file should be stored in the controller.

```
public bool? StoreInController { get; }
```

Property Value

[bool](#)[?]

true if the file should be stored in the controller; **false** if it should not; otherwise, **null** if not applicable.

Remarks

For files to which this attribute does not apply, the value is **null**.

Class IRCXProjectAPI.ProjectPropertyValue

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents a setting value for project properties.

```
public class IRCXProjectAPI.ProjectPropertyValue : RCXCommon.RCXSettingValue
```

Inheritance

[object](#)  ← [RCXCommon.RCXSettingValue](#) ← IRCXProjectAPI.ProjectPropertyValue

Inherited Members

[RCXCommon.RCXSettingValue.Value](#) , [RCXCommon.RCXSettingValue.ValueType](#) ,
[RCXCommon.RCXSettingValue.CanSet](#) , [RCXCommon.RCXSettingValue.Choices](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Interface IRCXProjectExplorerItemProvider

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Interface for providing items for the Project Explorer.

```
public interface IRCXProjectExplorerItemProvider
```

Properties

Id

Gets the Extension identifier.

```
string Id { get; }
```

Property Value

[string](#)

The unique identifier of the Extension.

ProjectExplorerRootItem

Gets the root item of the Project Explorer.

```
IRCXProjectExplorerItemProvider.Item ProjectExplorerRootItem { get; }
```

Property Value

[IRCXProjectExplorerItemProvider.Item](#)

The top-level item (root) displayed in the Project Explorer.

Methods

ExecuteProjectExplorerItemCommandAsync(string, object?)

Executes a command associated with a Project Explorer item asynchronously.

```
Task ExecuteProjectExplorerItemCommandAsync(string commandName, object? commandParameter  
= null)
```

Parameters

commandName [string](#) 

The command name.

commandParameter [object](#) 

The command parameter; optional.

Returns

[Task](#) 

A task that represents the asynchronous operation.

Class IRCXProjectExplorerItemProvider.Item

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents a tree item description.

```
public class IRCXProjectExplorerItemProvider.Item
```

Inheritance

[object](#)  ← IRCXProjectExplorerItemProvider.Item

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Properties

Caption

Gets or sets the caption of the tree item.

```
public RCXCaption Caption { get; set; }
```

Property Value

[RCXCaption](#)

The display caption shown for the item.

Children

Gets or sets the child items.

```
public IList<IRCXProjectExplorerItemProvider.Item>? Children { get; set; }
```

Property Value

[IList](#) <[IRCXProjectExplorerItemProvider.Item](#)>

A list of child [IRCXProjectExplorerItemProvider.Item](#) entries, or `null` if there are no children.

CommandName

Gets or sets the command name associated with the item.

```
public string CommandName { get; set; }
```

Property Value

[string](#)

The name of the command to execute when the item is invoked.

CommandParameter

Gets or sets the command parameter passed when the item's command is invoked.

```
public object? CommandParameter { get; set; }
```

Property Value

[object](#)

The parameter object for the command; may be `null`.

ContextMenuItems

Gets or sets the context menu items.

```
public IList<IRCXProjectExplorerItemProvider.MenuItem> ContextMenuItems { get; set; }
```

Property Value

[IList](#) <[IRCXProjectExplorerItemProvider.MenuItem](#)>

A list of context [IRCXProjectExplorerItemProvider.MenuItem](#) entries, or `null` if no context menu is provided.

Icon

Gets or sets the icon image data for the item.

```
public ImageSource? Icon { get; set; }
```

Property Value

[ImageSource](#)

The icon as an [ImageSource](#), or `null` if no icon is displayed.

SelectedIconData

Gets or sets the icon image data used when the item is selected.

```
public ImageSource? SelectedIconData { get; set; }
```

Property Value

[ImageSource](#)

The selected-state icon as an [ImageSource](#), or `null` if no alternate icon is used.

Class IRCXProjectExplorerItemProvider.MenuItem

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents a context menu item description.

```
public class IRCXProjectExplorerItemProvider.MenuItem
```

Inheritance

[object](#)  ← IRCXProjectExplorerItemProvider.MenuItem

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Properties

Caption

Gets or sets the caption of the menu item.

```
public RCXCaption Caption { get; set; }
```

Property Value

[RCXCaption](#)

The display caption. Typically shown as the text label of the menu item.

Children

Gets or sets the child menu items.

```
public IList<IRCXProjectExplorerItemProvider.MenuItem> Children { get; set; }
```

Property Value

[IList](#) <[IRCXProjectExplorerItemProvider.MenuItem](#)>

A list of child [IRCXProjectExplorerItemProvider.MenuItem](#) entries, or `null` if there are no children.

CommandName

Gets or sets the command name associated with the menu item.

```
public string CommandName { get; set; }
```

Property Value

[string](#)

The name of the command to execute.

CommandParameter

Gets or sets the command parameter passed when the command is invoked.

```
public object? CommandParameter { get; set; }
```

Property Value

[object](#)

The parameter object for the command; may be `null`.

Icon

Gets or sets the icon image data for the menu item.

```
public ImageSource? Icon { get; set; }
```

Property Value

[ImageSource](#)[↗]

The icon as an [ImageSource](#)[↗], or `null` if no icon is displayed.

Interface IRCXProjectFileProvider

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Interface for providing additional files for a project.

```
public interface IRCXProjectFileProvider
```

Properties

Extension

Gets the file extension.

```
string Extension { get; }
```

Property Value

[string](#)

The file extension, including the leading dot.

FileIcon

Gets the icon image data for the file.

```
ImageSource? FileIcon { get; }
```

Property Value

[ImageSource](#)

FileTypeNames

Gets the file type name.

```
string FileName { get; }
```

Property Value

[string](#)

The name of the file type. Must not contain whitespace characters.

FileNameCaption

Gets the display caption for the file type name.

```
RCXCaption FileNameCaption { get; }
```

Property Value

[RCXCaption](#)

Id

Gets the Extension identifier.

```
string Id { get; }
```

Property Value

[string](#)

The unique identifier of the Extension.

ProjectExplorerRootItemCaption

Gets the caption of the root category when using the default Project Explorer item.


```
RCXCaption ProjectExplorerRootItemCaption { get; }
```

Property Value

[RCXCaption](#)

ProjectExplorerRootItemIconData

Gets the icon image data of the root category when using the default Project Explorer item.

```
ImageSource? ProjectExplorerRootItemIconData { get; }
```

Property Value

[ImageSource](#)

UseDefaultProjectExplorerItem

Gets a value indicating whether the default Project Explorer item is used.

```
bool UseDefaultProjectExplorerItem { get; }
```

Property Value

[bool](#)

Methods

OpenAsync(string)

Asynchronously opens a file by executing the open command.

```
Task OpenAsync(string fileName)
```

Parameters

fileName [string](#)

The file name (or path) to open.

Returns

[Task](#)

A task representing the asynchronous operation.

WriteInitialContent(FileStream)

Writes the initial content when creating a file.

```
void WriteInitialContent(FileStream fileStream)
```

Parameters

fileStream [FileStream](#)

The file stream to write the initial content to.

Remarks

If an empty file is acceptable, return without writing anything.

Interface IRCXRobotManagerAPI

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Robot manager-related API

```
public interface IRCXRobotManagerAPI : INotifyPropertyChanged
```

Inherited Members

[INotifyPropertyChanged.PropertyChanged](#) 

Fields

MaxNumJoints

Gets the maximum number of joints.

```
public const int MaxNumJoints = 9
```

Field Value

[int](#) 

Properties

ArmOrientation

Gets the arm orientations.

```
IDictionary<string, object>? ArmOrientation { get; }
```

Property Value

[IDictionary](#)  <[string](#) , [object](#)  >

A dictionary of arm identifiers and their orientation values.

CurrentRobotNumber

Gets the robot number of the current robot.

```
int? CurrentRobotNumber { get; }
```

Property Value

[int](#)?

The current robot's number, or `null` if the controller is not connected.

JointPosition

Gets the joint positions.

```
IDictionary<IRCXRobotManagerAPI.RCXJogJointAxis, double>? JointPosition { get; }
```

Property Value

[IDictionary](#) <[IRCXRobotManagerAPI.RCXJogJointAxis](#), [double](#)>

A dictionary of joint axes and their position values.

JointState

Gets the current robot's joint states by axis.

```
IDictionary<IRCXRobotManagerAPI.RCXJogJointAxis, IRCXRobotManagerAPI.IRCXJointState>  
JointState { get; }
```

Property Value

[IDictionary](#) <[IRCXRobotManagerAPI.RCXJogJointAxis](#), [IRCXRobotManagerAPI.IRCXJointState](#)>

A dictionary of joint axes and their current states.

Remarks

Returns `null` if the controller is not connected or there is no current robot.

NumRobots

Gets the number of robots that have been added.

```
int? NumRobots { get; }
```

Property Value

[int](#)?

The count of added robots, or `null` if the controller is not connected.

PulsePosition

Gets the pulse positions.

```
IDictionary<IRCXRobotManagerAPI.RCXJogJointAxis, long>? PulsePosition { get; }
```

Property Value

[IDictionary](#) <[IRCXRobotManagerAPI.RCXJogJointAxis](#), [long](#)>

A dictionary of joint axes and their pulse values.

SelectedECPNumber

Gets the selected ECP coordinate number.

```
int? SelectedECPNumber { get; }
```

Property Value

[int?](#)

The selected ECP coordinate number, or **null** if none is selected.

SelectedLocalNumber

Gets the selected local coordinate number.

```
int? SelectedLocalNumber { get; }
```

Property Value

[int?](#)

The selected local coordinate number, or **null** if none is selected.

SelectedToolNumber

Gets the selected tool coordinate number.

```
int? SelectedToolNumber { get; }
```

Property Value

[int?](#)

The selected tool coordinate number, or **null** if none is selected.

SelectedVRTNumber

Gets the selected VRT number.

```
int? SelectedVRTNumber { get; }
```

Property Value

[int?](#)

The selected VRT number, or `null` if none is selected.

SysJogDistance

Gets or sets the system jog distance.

```
IRCXRobotManagerAPI.RCXJogDistance? SysJogDistance { get; set; }
```

Property Value

[IRCXRobotManagerAPI.RCXJogDistance?](#)

The system jog distance, or `null` if unavailable.

SysJogMode

Gets or sets the system jog mode.

```
IRCXRobotManagerAPI.RCXJogMode? SysJogMode { get; set; }
```

Property Value

[IRCXRobotManagerAPI.RCXJogMode?](#)

The system jog mode, or `null` if unavailable.

SysJogSpeed

Gets or sets the system jog speed.

```
IRCXRobotManagerAPI.RCXJogSpeed? SysJogSpeed { get; set; }
```

Property Value

[IRCXRobotManagerAPI.RCXJogSpeed?](#)

The system jog speed, or `null` if unavailable.

WorldPosition

Gets the position in world coordinates.

```
IDictionary<IRCXRobotManagerAPI.RCXJogCartesianAxis, double>? WorldPosition { get; }
```

Property Value

[IDictionary](#) <[IRCXRobotManagerAPI.RCXJogCartesianAxis](#), [double](#)>

A dictionary of axes and their position values.

Methods

CreateJoggerAsync()

Asynchronously creates a jog object.

```
Task<IRCXRobotManagerAPI.IRCXJogger> CreateJoggerAsync()
```

Returns

[Task](#) <[IRCXRobotManagerAPI.IRCXJogger](#)>

A task that returns the created jog object.

FreeAllJointsAsync(bool)

Asynchronously sets all switchable joints of the current robot to the specified state (freed or energized).

```
Task<RCXCommon.RCXResult> FreeAllJointsAsync(bool free)
```

Parameters

free [bool](#)

true to set all switchable joints to the freed (de-energized) state; otherwise, **false** to energize them.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code.

GetPartialRobotInfo(int)

Gets partial robot information.

```
(RCXCommon.RCXResult, IRCXRobotManagerAPI.RCXPartialRobotInfo) GetPartialRobotInfo(int robotNumber)
```

Parameters

robotNumber [int](#)

Robot number.

Returns

([RCXCommon.RCXResult](#), [IRCXRobotManagerAPI.RCXPartialRobotInfo](#))

A tuple containing the result code and the partial robot information.

GetRobotInfoAsync(int)

Asynchronously gets robot information.

```
Task<(RCXCommon.RCXResult, IRCXRobotManagerAPI.IRCXRobotInfo?)>  
GetRobotInfoAsync(int robotNumber)
```

Parameters

robotNumber [int](#)

Robot number.

Returns

[Task](#) <(RCXCommon.RCXResult, [IRCXRobotManagerAPI.IRCXRobotInfo](#))>

A task that returns a tuple containing the result code and the robot information.

GetRobotSettingsAsync(int, string)

Asynchronously gets the setting values for robot-related settings in a specified category.

```
Task<(RCXCommon.RCXResult, IDictionary<string, IRCXRobotManagerAPI.RobotSettingValue>?)>  
GetRobotSettingsAsync(int robotNumber, string category)
```

Parameters

robotNumber [int](#)

Robot number.

category [string](#)

The category name to retrieve.

Returns

[Task](#) <(RCXCommon.RCXResult, [IDictionary](#) <[string](#), [IRCXRobotManagerAPI.RobotSettingValue](#)>)>

A task that returns a tuple containing the result code and a dictionary of setting values, keyed by setting name.

GetRobotSettingsCategories()

Gets the list of category names for robot-related settings.

```
IEnumerable<string> GetRobotSettingsCategories()
```

Returns

[IEnumerable](#) <[string](#)>

An enumerable collection of category names.

SelectECPAsync(int)

Asynchronously selects an ECP coordinate.

```
Task<RCXCommon.RCXResult> SelectECPAsync(int number)
```

Parameters

number [int](#)

ECP coordinate number; specify 0 to clear the selection.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code.

SelectLocalAsync(int)

Asynchronously selects a local coordinate.

```
Task<RCXCommon.RCXResult> SelectLocalAsync(int number)
```

Parameters

number [int](#)

Local coordinate number; specify 0 to clear the selection.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code.

SelectToolAsync(int)

Asynchronously selects a tool coordinate.

```
Task<RCXCommon.RCXResult> SelectToolAsync(int number)
```

Parameters

number [int](#)

Tool coordinate number; specify 0 to clear the selection.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code.

SelectVRTAsync(int)

Asynchronously selects a VRT.

```
Task<RCXCommon.RCXResult> SelectVRTAsync(int number)
```

Parameters

number [int](#)

VRT number; specify 0 to clear the selection.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code.

SetCurrentRobotAsync(int)

Asynchronously changes the current robot.

```
Task<RCXCommon.RCXResult> SetCurrentRobotAsync(int robotNumber)
```

Parameters

robotNumber [int](#)

The robot number to set as current.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns an [RCXCommon.RCXResult](#) indicating the result code of the operation.

SetRobotInfoAsync(int, IRCXRobotInfo)

Asynchronously updates robot information.

```
Task<RCXCommon.RCXResult> SetRobotInfoAsync(int robotNumber,  
IRCXRobotManagerAPI.IRCXRobotInfo info)
```

Parameters

robotNumber [int](#)

Robot number.

info [IRCXRobotManagerAPI.IRCXRobotInfo](#)

Robot information to apply.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code.

Remarks

Only the following fields can be changed:

- Name
- Serial number
- Enable/disable state of each axis

Executing this operation will restart the controller.

SetRobotSettingsAsync(int, string, IDictionary<string, RobotSettingValue>)

Asynchronously sets (updates) the setting values for robot-related settings in a specified category.

```
Task<RCXCommon.RCXResult> SetRobotSettingsAsync(int robotNumber, string category,  
IDictionary<string, IRCXRobotManagerAPI.RobotSettingValue> settings)
```

Parameters

robotNumber [int](#)

Robot number.

category [string](#)

The category name to update.

settings [IDictionary](#) <[string](#), [IRCXRobotManagerAPI.RobotSettingValue](#)>

A dictionary of setting values, keyed by setting name, to be applied.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code.

Interface IRCXRobotManagerAPI.IRCXJogger

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents a jog object.

```
public interface IRCXRobotManagerAPI.IRCXJogger : IDisposable
```

Inherited Members

[IDisposable.DisposeAsync\(\)](#) 

Properties

IsValid

Gets a value indicating whether the object is valid.

```
bool IsValid { get; }
```

Property Value

[bool](#) 

true if the object is valid; otherwise, **false**.

ShouldKeepJogging

Gets or sets a function that determines whether continuous jogging should continue.

```
Func<bool>? ShouldKeepJogging { get; set; }
```

Property Value

[Func](#)  [<bool](#)  [>](#)

A Function that returns **true** to keep jogging; otherwise, **false**.

Methods

CancelJog()

Cancels the current jog operation.

```
void CancelJog()
```

StartCartesianJogAsync(RCXJogCartesianAxis, bool, bool)

Asynchronously starts a jog operation by specifying a Cartesian axis.

```
Task<(RCXCommon.RCXResult, string)>  
StartCartesianJogAsync(IRCXRobotManagerAPI.RCXJogCartesianAxis cartesianAxis, bool  
oppositeDirection = false, bool isContinuous = false)
```

Parameters

cartesianAxis [IRCXRobotManagerAPI.RCXJogCartesianAxis](#)

The target Cartesian axis.

oppositeDirection [bool](#)

Whether to jog in the opposite direction.

isContinuous [bool](#)

Whether to jog continuously; otherwise, step mode.

Returns

[Task](#) <(RCXCommon.RCXResult, [string](#))>

A task that returns a tuple consisting of the result code and the error message.

StartJointJogAsync(RCXJogJointAxis, bool, bool)

Asynchronously starts a jog operation by specifying a joint.


```
Task<(RCXCommon.RCXResult, string)> StartJointJogAsync(IRCXRobotManagerAPI.RCXJogJointAxis  
jointAxis, bool oppositeDirection = false, bool isContinuous = false)
```

Parameters

jointAxis [IRCXRobotManagerAPI.RCXJogJointAxis](#)

The target joint axis.

oppositeDirection [bool](#)

Whether to jog in the opposite direction.

isContinuous [bool](#)

Whether to jog continuously; otherwise, step mode.

Returns

[Task](#) <(RCXCommon.RCXResult, [string](#))>

A task that returns a tuple consisting of the result code and the error message.

Interface IRCXRobotManagerAPI.IRCXJointState

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents the joint state.

```
public interface IRCXRobotManagerAPI.IRCXJointState
```

Properties

CanFree

Gets whether the joint can be freed.

```
bool CanFree { get; }
```

Property Value

[bool](#)

true if the joint can be freed; otherwise, **false**.

IsEnabled

Gets whether the joint is enabled.

```
bool IsEnabled { get; }
```

Property Value

[bool](#)

true if the joint is enabled; otherwise, **false**.

IsFreed

Gets whether the joint is freed.

```
bool IsFreed { get; }
```

Property Value

[bool](#)

true if the joint is freed; otherwise, **false**.

Methods

FreeAsync(bool)

Asynchronously sets the joint to the freed (de-energized) state.

```
Task<RCXCommon.RCXResult> FreeAsync(bool free)
```

Parameters

free [bool](#)

true to set the joint to the freed (de-energized) state; otherwise, **false** to energize it.

Returns

[Task](#) <[RCXCommon.RCXResult](#)>

A task that returns the result code.

Interface IRCXRobotManagerAPI.IRCXRobotInfo

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Provides robot information.

```
public interface IRCXRobotManagerAPI.IRCXRobotInfo
```

Remarks

Retrieval may take some time.

Properties

AxisIsEnabled

Gets or init-only sets the per-axis enablement states.

```
bool?[] AxisIsEnabled { get; init; }
```

Property Value

[bool](#)?[]

An array of per-axis enablement states where each element indicates whether the corresponding axis is enabled: **true** = enabled, **false** = disabled, **null** = unknown or not applicable.

LengthJ1J2

Gets or init-only sets the combined arm length for joints J1 and J2.

```
double? LengthJ1J2 { get; init; }
```

Property Value

[double](#)?

The combined arm length for J1 and J2; **null** if unavailable.

LengthX

Gets or init-only sets the arm length along the X axis.

```
double? LengthX { get; init; }
```

Property Value

[double](#)?

The X-axis arm length; **null** if unavailable.

LengthY

Gets or init-only sets the arm length along the Y axis.

```
double? LengthY { get; init; }
```

Property Value

[double](#)?

The Y-axis arm length; **null** if unavailable.

LengthZ

Gets or init-only sets the arm length along the Z axis.

```
double? LengthZ { get; init; }
```

Property Value

[double](#)?

The Z-axis arm length; `null` if unavailable.

MaxPayload

Gets or init-only sets the maximum payload.

```
double? MaxPayload { get; init; }
```

Property Value

[double](#)?

The maximum payload capacity; `null` if unavailable.

ModelName

Gets or init-only sets the robot model name.

```
string ModelName { get; init; }
```

Property Value

[string](#)

The robot model name.

Name

Gets or sets the robot name.

```
string Name { get; set; }
```

Property Value

[string](#)

The robot name.

Reach

Gets or init-only sets the reach (maximum radial distance).

```
double? Reach { get; init; }
```

Property Value

[double](#)?

The reach length; `null` if unavailable.

SAxisUserName

Gets or sets the user model name for the S axis.

```
string? SAxisUserName { get; set; }
```

Property Value

[string](#)

The S-axis user model name; `null` if unavailable or not set.

SerialNumber

Gets or sets the robot serial number.

```
string SerialNumber { get; set; }
```

Property Value

[string](#)

The robot serial number.

TAxisUserName

Gets or sets the user model name for the T axis.

```
string? TAxisUserModelName { get; set; }
```

Property Value

[string](#) 

The T-axis user model name; **null** if unavailable or not set.

TypeName

Gets or init-only sets the type name.

```
string TypeName { get; init; }
```

Property Value

[string](#) 

The type name.

Methods

AreChangeablesEqualTo(IRCXRobotInfo)

変更可能な情報が同じか否かを返します。

```
bool AreChangeablesEqualTo(IRCXRobotManagerAPI.IRCXRobotInfo other)
```

Parameters

other [IRCXRobotManagerAPI.IRCXRobotInfo](#)

別のロボット情報

Returns

[bool](#) 

同じか否かのフラグ

Copy()

Gets a copy of this robot information.

```
IRCXRobotManagerAPI.IRCXRobotInfo Copy()
```

Returns

[IRCXRobotManagerAPI.IRCXRobotInfo](#)

A copy of the robot information.

Enum IRCXRobotManagerAPI.RCXJogCartesianAxis

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Jog Cartesian axis specification.

```
public enum IRCXRobotManagerAPI.RCXJogCartesianAxis
```

Fields

R = 6

S = 7

T = 8

U = 3

V = 4

W = 5

X = 0

Y = 1

Z = 2

Enum IRCXRobotManagerAPI.RCXJogDistance

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Jog step amount (distance increment).

```
public enum IRCXRobotManagerAPI.RCXJogDistance
```

Fields

Continuous = 3

Long = 2

Middle = 1

Short = 0

Enum IRCXRobotManagerAPI.RCXJogJointAxis

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Jog joint axis specification.

```
public enum IRCXRobotManagerAPI.RCXJogJointAxis
```

Fields

J1 = 0

J2 = 1

J3 = 2

J4 = 3

J5 = 4

J6 = 5

J7 = 6

J8 = 7

J9 = 8

Enum IRCXRobotManagerAPI.RCXJogMode

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Jog mode specification.

```
public enum IRCXRobotManagerAPI.RCXJogMode
```

Fields

```
ECP = 4
```

```
Joint = 0
```

```
Local = 2
```

```
Tool = 3
```

```
World = 1
```

Enum IRCXRobotManagerAPI.RCXJogSpeed

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Jog speed.

```
public enum IRCXRobotManagerAPI.RCXJogSpeed
```

Fields

Fast = 1

Slow = 0

Class IRCXRobotManagerAPI.RCXPartialRobotInfo

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Provides a partial subset of robot information.

```
public class IRCXRobotManagerAPI.RCXPartialRobotInfo
```

Inheritance

[object](#)  ← IRCXRobotManagerAPI.RCXPartialRobotInfo

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Fields

ModelName

The robot model name.

```
public string ModelName
```

Field Value

[string](#) 

Name

The robot name.

```
public string Name
```

Field Value

[string](#) 

SerialNumber

The robot serial number.

```
public string SerialNumber
```

Field Value

[string](#) 

Class IRCXRobotManagerAPI.RobotSettingValue

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents the setting values related to robot configuration.

```
public class IRCXRobotManagerAPI.RobotSettingValue : RCXCommon.RCXSettingValue
```

Inheritance

[object](#)  ← [RCXCommon.RCXSettingValue](#) ← IRCXRobotManagerAPI.RobotSettingValue

Inherited Members

[RCXCommon.RCXSettingValue.Value](#) , [RCXCommon.RCXSettingValue.ValueType](#) ,
[RCXCommon.RCXSettingValue.CanSet](#) , [RCXCommon.RCXSettingValue.Choices](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Interface IRCXUserControlViewModel

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Interface for providing a view model for a UserControl hosted in a docking window.

```
public interface IRCXUserControlViewModel : IDisposable
```

Inherited Members

[IDisposable.Dispose\(\)](#) 

Properties

Id

Gets the Extension identifier.

```
string Id { get; }
```

Property Value

[string](#) 

The unique identifier of the Extension.

KeepOpenWhenProjectClosing

Gets a value indicating whether the window remains open when the current project is closing.

```
bool KeepOpenWhenProjectClosing { get; }
```

Property Value

[bool](#) 

true to keep the window open; otherwise, **false**.

ViewModelId

Gets the ViewModel identifier.

```
string ViewModelId { get; }
```

Property Value

[string](#)

Remarks

Assign a unique ID per window when multiple windows are used.

WindowCaption

Gets or sets the window caption.

```
RCXCaption WindowCaption { get; set; }
```

Property Value

[RCXCaption](#)

WindowIcon

Gets or sets the window icon.

```
ImageSource? WindowIcon { get; set; }
```

Property Value

[ImageSource](#)

Methods

CloseAsync()

Asynchronously performs the closing process for the window.

```
Task<bool> CloseAsync()
```

Returns

[Task](#) <[bool](#)>

A task that returns `true` to continue; otherwise, `false`.

Copy()

Performs the copy process.

```
void Copy()
```

Cut()

Performs the cut process.

```
void Cut()
```

Paste()

Performs the paste process.

```
void Paste()
```

Redo()

Performs the redo process.

```
void Redo()
```

Reload()

Performs the content reload process.

```
void Reload()
```

SaveAsync()

Asynchronously performs the save process.

```
Task<bool> SaveAsync()
```

Returns

[Task](#) <[bool](#)>

A task that returns `true` to continue; otherwise, `false`.

SelectAll()

Performs the select-all process.

```
void SelectAll()
```

ShowHelp()

Performs the process to display help.

```
void ShowHelp()
```

Undo()

Performs the undo process.

```
void Undo()
```

WindowCreated()

Asynchronously performs post-creation processing after the window has been created.

```
Task WindowCreated()
```

Returns

[Task](#)

A task that represents the asynchronous operation.

Interface IRCXWindowAPI

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Provides window-related API.

```
public interface IRCXWindowAPI
```

Methods

GetContentState(IRCXUserControlViewModel)

Gets the content state.

```
IRCXWindowAPI.ContentState GetContentState(IRCXUserControlViewModel userControlViewModel)
```

Parameters

userControlViewModel [IRCXUserControlViewModel](#)

The view model of the user control.

Returns

[IRCXWindowAPI.ContentState](#)

The content state.

SetContentState(IRCXUserControlViewModel, ContentState)

Sets the content state.

```
void SetContentState(IRCXUserControlViewModel userControlViewModel,  
IRCXWindowAPI.ContentState state)
```

Parameters

userControlViewModel [IRCXUserControlViewModel](#)

The view model of the user control.

state [IRCXWindowAPI.ContentState](#)

The content state.

ShowDockingWebViewWindowAsync(WebViewInfo)

Asynchronously opens a docking window that displays web content using the specified [IRCXWindowAPI.WebViewInfo](#).

Task `ShowDockingWebViewWindowAsync(IRCXWindowAPI.WebViewInfo webViewInfo)`

Parameters

webViewInfo [IRCXWindowAPI.WebViewInfo](#)

Definition and configuration for the WebView to be displayed.

Returns

[Task](#) 

A task that represents the asynchronous operation.

ShowDockingWindowAsync(IRCXUserControlViewModel, UserControl)

Asynchronously shows a docking window hosting the specified user control and its view model.

Task `ShowDockingWindowAsync(IRCXUserControlViewModel userControlViewModel, UserControl userControl)`

Parameters

userControlViewModel [IRCXUserControlViewModel](#)

The view model of the user control.

userControl [UserControl](#)

The user control.

Returns

[Task](#)

A task that represents the asynchronous operation.

ShowMessageBox(RCXCaption, RCXCaption, ButtonType, IconType, Window?)

Shows a message box.

```
IRCXWindowAPI.ResponseType ShowMessageBox(RCXCaption title, RCXCaption message,  
IRCXWindowAPI.ButtonType buttonType, IRCXWindowAPI.IconType iconType, Window? owner = null)
```

Parameters

title [RCXCaption](#)

The title of the message box.

message [RCXCaption](#)

The message text.

buttonType [IRCXWindowAPI.ButtonType](#)

The type of buttons to display.

iconType [IRCXWindowAPI.IconType](#)

The type of icon to display.

owner [Window](#)

The owner window.

Returns

[IRCXWindowAPI.ResponseType](#)

The response selected by the user.

Enum IRCXWindowAPI.ButtonType

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Specifies the set of buttons to display in a message box.

```
public enum IRCXWindowAPI.ButtonType
```

Fields

```
Abort_Retry_Ignore = 5
```

```
Delete_Exclude_Cancel = 6
```

```
OK = 0
```

```
OK_Cancel = 1
```

```
Retry_Cancel = 4
```

```
Yes_No = 2
```

```
Yes_No_Cancel = 3
```

Enum IRCXWindowAPI.ContentState

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents flags that indicate the current content state.

[Flags]

```
public enum IRCXWindowAPI.ContentState
```

Fields

CanCopy = 2

Copy operation is available.

CanCut = 4

Cut operation is available.

CanPaste = 8

Paste operation is available.

CanRedo = 64

Redo operation is available.

CanSelectAll = 16

Select All operation is available.

CanUndo = 32

Undo operation is available.

IsDirty = 1

Content has been modified.

None = 0

Enum IRCXWindowAPI.IconType

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Specifies the icon to display in a message box.

```
public enum IRCXWindowAPI.IconType
```

Fields

Error = 1

Information = 3

None = 0

Question = 4

Warning = 2

Enum IRCXWindowAPI.ResponseType

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents the user's response in a message box.

```
public enum IRCXWindowAPI.ResponseType
```

Fields

Cancel = 0

No = 1

Yes = 2

Class IRCXWindowAPI.WebMessage

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents the common structure of messages exchanged with the web side.

```
public class IRCXWindowAPI.WebMessage
```

Inheritance

[object](#)  ← IRCXWindowAPI.WebMessage

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

WebMessage(string, string?)

Initializes a new instance of the [IRCXWindowAPI.WebMessage](#) class with the specified command and optional parameter.

```
public WebMessage(string command, string? parameter = null)
```

Parameters

command [string](#) 

The command to execute.

parameter [string](#) 

An optional parameter for the command.

Properties

Command

Gets the command to execute.

```
public string Command { get; }
```

Property Value

[string](#)

Parameter

Gets and sets an optional parameter for the command.

```
public string? Parameter { get; set; }
```

Property Value

[string](#)

Methods

ToJson()

Serializes the current object to a JSON string.

```
public string ToJson()
```

Returns

[string](#)

A JSON string that represents the current object.

Class IRCXWindowAPI.WebViewInfo

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Contains definition and configuration details for a WebView component.

```
public class IRCXWindowAPI.WebViewInfo
```

Inheritance

[object](#)  ← IRCXWindowAPI.WebViewInfo

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

WebViewInfo(Func<string, uint?, uint?, Uri?>, string, string, RCXCaption, ImageSource?)

Initializes a new instance of [IRCXWindowAPI.WebViewInfo](#) with the URI source delegate, extension and view-model identifiers, and window presentation details.

```
public WebViewInfo(Func<string, uint?, uint?, Uri?> sourceBuilder, string extensionId, string viewModelId, RCXCaption windowCaption, ImageSource? windowIcon = null)
```

Parameters

sourceBuilder [Func](#)  <[string](#) , [uint](#) ?, [uint](#) ?, [Uri](#)  >

Function that returns the [Uri](#)  for WebView content.

extensionId [string](#) 

Extension identifier.

viewModelId [string](#) 

View-model identifier.

`windowCaption` [RCXCaption](#)

Window caption.

`windowIcon` [ImageSource](#)

Optional window icon.

Fields

AddHostObjectsHook

ブリッジ登録フック

```
public Action<WebView2>? AddHostObjectsHook
```

Field Value

[Action](#) <[WebView2](#) >

CloseScript

Script executed when the window closes.

```
public string? CloseScript
```

Field Value

[string](#)

ExtensionId

Extension ID

```
public string ExtensionId
```

Field Value

[string](#)

ImportHelperDocumentRoot

Document root path for the local HTTP server.

```
public string? ImportHelperDocumentRoot
```

Field Value

[string](#)

Remarks

Specify when using dynamic module imports.

KeepOpenWhenProjectClosing

Indicates whether the window remains open when the current project is closed.

```
public bool KeepOpenWhenProjectClosing
```

Field Value

[bool](#)

`true` to keep the window open; otherwise, `false`.

SaveScript

Script executed when saving.

```
public string? SaveScript
```

Field Value

[string](#)

SourceBuilder

Provides a function that returns a URI for WebView content.

```
public Func<string, uint?, uint?, Uri?> SourceBuilder
```

Field Value

[Func](#) <[string](#), [uint](#)?, [uint](#)?, [Uri](#)>

Remarks

The function takes the following arguments:

1. Current display language
2. Port number when the local HTTP server is started
3. Port number when the web application is started If the returned URI is `null`, no content will be displayed.

ViewModelId

ViewModel ID.

```
public string ViewModelId
```

Field Value

[string](#)

Remarks

If the extension uses multiple WebView windows, assign a unique ID to each.

WebAppCommand

Path to the web application startup command.

`public string?` WebAppCommand

Field Value

[string](#)

Remarks

If not a full path, it will be searched in the PATH environment variable.

WebAppCommandArguments

Command-line argument string for the web application.

`public string?` WebAppCommandArguments

Field Value

[string](#)

Remarks

`\({port}</code>` is replaced with the port number, and `<code>\){lang}` is replaced with the display language.

WebAppDocumentRoot

Document root path for the web application.

`public string?` WebAppDocumentRoot

Field Value

[string](#)

WindowCaption

Window caption.

```
public RCXCaption WindowCaption
```

Field Value

[RCXCaption](#)

WindowIcon

Window icon.

```
public ImageSource? WindowIcon
```

Field Value

[ImageSource](#)

Class RCXCaption

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Represents a caption specified by number or text.

```
public class RCXCaption : DependencyObject
```

Inheritance

[object](#)  ← [DispatcherObject](#)  ← [DependencyObject](#)  ← RCXCaption

Inherited Members

[DependencyObject.ClearValue\(DependencyProperty\)](#)  ,
[DependencyObject.ClearValue\(DependencyPropertyKey\)](#)  ,
[DependencyObject.CoerceValue\(DependencyProperty\)](#)  , [DependencyObject.Equals\(object\)](#)  ,
[DependencyObject.GetHashCode\(\)](#)  , [DependencyObject.GetLocalValueEnumerator\(\)](#)  ,
[DependencyObject.GetValue\(DependencyProperty\)](#)  ,
[DependencyObject.InvalidateProperty\(DependencyProperty\)](#)  ,
[DependencyObject.OnPropertyChanged\(DependencyPropertyChangedEventArgs\)](#)  ,
[DependencyObject.ReadLocalValue\(DependencyProperty\)](#)  ,
[DependencyObject.SetCurrentValue\(DependencyProperty, object\)](#)  ,
[DependencyObject.SetValue\(DependencyProperty, object\)](#)  ,
[DependencyObject.SetValue\(DependencyPropertyKey, object\)](#)  ,
[DependencyObject.ShouldSerializeProperty\(DependencyProperty\)](#)  ,
[DependencyObject.DependencyObjectType](#)  , [DependencyObject.IsSealed](#)  ,
[DispatcherObject.Dispatcher](#)  , [object.Equals\(object, object\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Constructors

RCXCaption()

Initializes a new instance of the [RCXCaption](#) class with a default placeholder text.

```
public RCXCaption()
```

RCXCaption(string)

Initializes a new instance of the [RCXCaption](#) class with the specified caption text.

```
public RCXCaption(string text)
```

Parameters

text [string](#) 

The caption text.

RCXCaption(string, int)

Initializes a new instance of the [RCXCaption](#) class with the specified ID and caption number.

```
public RCXCaption(string id, int captionNumber)
```

Parameters

id [string](#) 

The Extensions ID.

captionNumber [int](#) 

The caption number.

Fields

Empty

Represents an empty caption.

```
public static readonly RCXCaption Empty
```

Field Value

[RCXCaption](#)

Id

Extension identifier.

```
public string? Id
```

Field Value

[string](#)

Tag

The caption specification type.

```
public RCXCaption.TypeTag Tag
```

Field Value

[RCXCaption.TypeTag](#)

Value

The caption content (number or text).

```
public object Value
```

Field Value

[object](#)

Enum RCXCaption.TypeTag

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Specifies how the caption is defined.

```
public enum RCXCaption.TypeTag
```

Fields

`CaptionNumber = 0`

Specified by its number.

`Text = 1`

Specified by the text.

Class RCXCommon

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Common definitions for interfaces used by RC+ Extensions.

```
public static class RCXCommon
```

Inheritance

[object](#)  ← RCXCommon

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Enum RCXCommon.RCXResult

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Common result codes.

```
public enum RCXCommon.RCXResult
```

Fields

AlreadyEncrypted = 121

Already encrypted.

AtLeastOneProgram = 103

At least one program is required.

AtLeastOneVisibleTab = 104

At least one visible tab is required.

AtLeastOneVisibleView = 105

At least one visible view is required.

BackgroundTasksNotAllowed = 203

Background tasks are not allowed.

BadArgument = 304

Invalid argument.

BadCartesianAxis = 209

Invalid Cartesian axis specification.

BadCoordinateNumber = 212

Invalid coordinate number.

BadIOKind = 217

Invalid I/O kind.

BadJogMode = 210

Invalid jog mode.

BadJointAxis = 207

Invalid joint axis specification.

BadLineNumber = 237

Invalid line number.

BadPointData = 227

Invalid point data.

BadPointLabelFormat = 229

Invalid point label format.

BadPortNumber = 213

Invalid port number.

BridgeDLLError = 300

Bridge DLL access error.

BuildFailed = 205

Build failed.

Canceled = 2

Canceled.

CannotAddAnyMore = 225

Cannot add any more.

CannotConvertToSpecifiedType = 236

Cannot convert to the specified type.

CannotCreateFolder = 113

Cannot create folder.

CannotCreateSuchKind = 224

Cannot create such kind.

CannotDeleteCurrentProject = 116

Cannot delete the currently opened project.

CannotPlaceBreakPoint = 238

Cannot place a breakpoint on this line.

CategoryNotFound = 101

Category not found.

ConnectionInUse = 226

Connection info is in use.

ControllerDisconnected = 200

Controller is disconnected.

DimesionMismatch = 235

Array index dimension mismatch.

DisabledJoint = 208

Joint is disabled.

DuplicatedTabOrder = 106

Duplicate tab order.

FailedToLaunchProgram = 305

Failed to launch external program.

Failure = 1

Failure.

FixedName = 119

Name is fixed (cannot be renamed).

`IOLabelNotExist = 218`

The I/O label does not exist.

`InvalidIPAddress = 110`

Invalid IP address.

`InvalidJoggerInstance = 239`

Invalid Jogger instance.

`InvalidOptionName = 109`

Invalid option name.

`InvalidProjectName = 114`

Invalid project name.

`InvalidRobotNumber = 107`

Invalid robot number.

`InvalidSession = 102`

Session not started or ID is invalid.

`InvalidValue = 3`

Invalid value.

`IsJoggingAlready = 206`

Jogging is already in progress.

`MainFunctionsCannotRunInBackground = 204`

Main functions cannot run in the background.

`ModelNotSupported = 108`

Model not supported by the controller.

`NameAlreadyUsed = 223`

Name is already in use.

NoCurrentRobot = 211

No current robot set.

NoSuchConnection = 220

No connection with that name exists.

NoSuchFunction = 303

Function is not registered.

NoSuchPoint = 232

No point with that number exists.

NotEncryptable = 120

Not encryptable.

NotEncrypted = 122

Not encrypted.

NotEthernetConnection = 221

Not an Ethernet connection.

NotForCurrentRobot = 233

Not for the current robot.

NotPermitted = 201

Not permitted.

NotPermittedUnderCurrentConditions = 202

Not permitted under current conditions.

NotProjectFolder = 117

Not a project folder.

NotVirtualConnection = 222

Not a virtual controller connection.

`NotVirtualIO = 215`

Not a virtual I/O.

`NotYetSupported = 5`

Not yet supported.

`OutOfRange = 4`

Value out of range.

`PasswordRequiredForGlobalIP = 111`

Password required to use a global IP.

`PathExists = 115`

A file or folder with the same name as the project folder already exists.

`PointDescriptionExceedsMaxBytes = 231`

Point description exceeds the maximum allowed bytes.

`PointLabelExceedsMaxBytes = 230`

Point label exceeds the maximum allowed bytes.

`PointNumberOutOfRange = 228`

Point number out of range.

`ProgramExitWithError = 306`

External program exited with error.

`ProjectNotFound = 118`

Project not found.

`ProjectNotOpened = 100`

Project is not opened.

`ProjectTemplateNotFound = 112`

Matching project template not found.

ReservedWord = 219

Reserved word.

SecurityPermissionDenied = 6

Security permission denied.

SharedMemoryError = 301

Shared memory access error.

Success = 0

Success.

Timeout = 302

Timeout.

UnexpectedTaskState = 234

Unexpected task state.

UnsavedDataExists = 216

Unsaved data exists.

UnsupportedDataType = 214

Unsupported data type.

WrongPassword = 123

Wrong password.

Class RCXCommon.RCXSettingValue

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Base class for setting values.

```
public class RCXCommon.RCXSettingValue
```

Inheritance

[object](#)  ← RCXCommon.RCXSettingValue

Derived

[IRCXControllerAPI.ControllerSettingValue](#), [IRCXPreferencesAPI.PreferenceValue](#),
[IRCXProjectAPI.ProjectPropertyValue](#), [IRCXRobotManagerAPI.RobotSettingValue](#)

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Fields

CanSet

Indicates whether the value can be changed.

```
public bool CanSet
```

Field Value

[bool](#) 

Choices

The selectable choices for the value.

```
public IEnumerable<object>? Choices
```

Field Value

[IEnumerable](#) <[object](#)>

Remarks

If not [null](#), any change must use one of the provided choices.

Value

The value.

```
public object? Value
```

Field Value

[object](#)

ValueType

The value's type in CLR.

```
public Type ValueType
```

Field Value

[Type](#)

Delegate RCXExternalFunction

Namespace: [Epson.RoboticsShared.ExtensionsAPI](#)

Assembly: Epson.RoboticsShared.ExtensionsAPI.dll

Delegate for invoking an external function.

```
public delegate (RCXCommon.RCXResult, string) RCXExternalFunction(string command,
    IList<string> parameters)
```

Parameters

command [string](#)

The command name to execute.

parameters [IList](#) <[string](#)>

The list of string arguments passed to the function.

Returns

([RCXCommon.RCXResult](#), [string](#))

A (T1, T2) containing:

- [RCXCommon.RCXResult](#) — the result code.
- [string](#) — the output text.

Remarks

RC+ Extensions must export this delegate for external functions and provide a function name via [ExportMetadata](#). Example:

```
[Export(typeof(RCXExternalFunction))]  
[ExportMetadata("Name", "MyFunction")]  
public RCXExternalFunction MyFunction => (commandName, arguments) => ...
```